

Aprentatge Automàtic II: Fonaments de l'Aprentatge Automàtic i Deep Learning

Codi: 104871 - Grau en Estadística Aplicada

Víctor Navas Portella

Segon quadrimestre- Curs 2025/2026

UAB
**Universitat Autònoma
de Barcelona**

Índex

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

- 1 Context: AI, ML i DL
- 2 Taxonomia del Machine Learning
- 3 Aprentatge Supervisat
- 4 Inferència Estadística: Pèrdua i Versemblança
- 5 Avaluació de Models
- 6 Generalització
- 7 Connexió amb Xarxes Neuronals
- 8 Perceptró (Classificació lineal)
- 9 Regressió Logística (GLM)
- 10 Regressió (sortida contínua)
- 11 La Neurona Artificial
- 12 Playground (laboratori visual)
- 13 Arquitectura: De la Neurona a la Capa
 - Shallow networks
 - El Teorema de l'Aproximació Universal (UA)
 - Xarxes neuronals profundes

Definicions i Jerarquia

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Sovint s'utilitzen els termes indistintament, però representen conceptes niats:

- **Intel·ligència Artificial (IA):** El camp general. Sistemes que simulen comportament intel·ligent (inclou sistemes basats en regles, lògica, cerca...).
- **Aprenentatge Automàtic (ML):** Un subconjunt de la IA. Es basa en ajustar models matemàtics a dades observades per prendre decisions, en lloc de programar regles explícites.
- **Aprenentatge Profund (DL):** Un subconjunt del ML que utilitza xarxes neuronals profundes. És l'estat de l'art actual en visió, text i àudio.

Sovint s'utilitzen els termes indistintament, però representen conceptes niats:

- **Intel·ligència Artificial (IA):** El camp general. Sistemes que simulen comportament intel·ligent (inclou sistemes basats en regles, lògica, cerca...).
- **Aprenentatge Automàtic (ML):** Un subconjunt de la IA. Es basa en ajustar **models matemàtics** a dades observades per prendre decisions, en lloc de programar regles explícites.
- **Aprenentatge Profund (DL):** Un subconjunt del ML que utilitza **xarxes neuronals profundes**. És l'estat de l'art actual en visió, text i àudio.

Definicions i Jerarquia

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Sovint s'utilitzen els termes indistintament, però representen conceptes niats:

- **Intel·ligència Artificial (IA):** El camp general. Sistemes que simulen comportament intel·ligent (inclou sistemes basats en regles, lògica, cerca...).
- **Aprenentatge Automàtic (ML):** Un subconjunt de la IA. Es basa en ajustar **models matemàtics** a dades observades per prendre decisions, en lloc de programar regles explícites.
- **Aprenentatge Profund (DL):** Un subconjunt del ML que utilitza **xarxes neuronals profundes**. És l'estat de l'art actual en visió, text i àudio.

Des del punt de vista estadístic, assumim un procés generador de dades desconegut.

El Parell Aleatori

Sigui (X, Y) un parell aleatori generat per una distribució conjunta:

$$(X, Y) \sim f(X, Y)$$

D'aquí sorgeixen els tres objectes d'estudi fonamentals:

- ❶ **Model Marginal** $f(X)$: Estudi de la distribució de les dades (sense etiquetes).
- ❷ **Model Condicional** $f(Y | X)$: Predicció de la resposta donada l'entrada.
- ❸ **Model Conjunt** $f(X, Y)$: Descripció completa del procés generador.

Podem classificar gairebé qualsevol algorisme segons dos criteris independents:

1. Quines dades tenim? (Tipus d'aprenentatge)

- **Supervisat:** Observem parelles (X, Y) .
- **No Supervisat:** Només observem X .

2. Què modelem? (Objectiu estadístic)

- **Discriminatiu:** Modelem directament la frontera o $f(Y | X)$.
- **Generatiu:** Modelem la distribució de les dades $f(X)$ o $f(X, Y)$.

Mapa de Mètodes de Machine Learning

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

	Discriminatiu Aprendre la frontera ($P(Y X)$)	Generatiu Aprendre la distribució ($P(X, Y)$)
Supervisat (Dades etiquetades)	Regressió Logística, SVM, Xarxes Neuronals , Arbres de Decisió	Naive Bayes, LDA, Hidden Markov Models (HMM)
No Supervisat (Sense etiquetes)	Aprentatge de Representacions: PCA, K-Means, t-SNE	GMM, Variational Autoencoders (VAE), GANs, Models de Difusió

- **Deep Learning:** És transversal. Una CNN és discriminativa, però un VAE és generatiu.
- **Nota:** El clúster (K-Means) busca estructures, no necessàriament probabilitats.

L'objectiu és inferir una funció $f: \mathcal{X} \rightarrow \mathcal{Y}$ a partir d'un conjunt d'entrenament (*Training Set*).

$$\mathcal{D} = \{(\mathbf{x}^{(k)}, y^{(k)})\}_{k=1}^N$$

On:

- $\mathbf{x}^{(k)} \in \mathbb{R}^D$: Vector de característiques (*inputs*) de la mostra k .
- $y^{(k)} \in \mathcal{Y}$: Variable resposta (*target*) de la mostra k .
- $\mathcal{Y}$: Espai de sortida (ex: $\{0, 1\}$ en classificació, $\mathbb{R}$ en regressió).

La naturalesa de l'espai $\mathcal{Y}$ defineix el tipus de problema:

1. Classificació

- $\mathcal{Y}$ és un conjunt finit (categories).
- Ex: $\{0, 1\}$ o $\{gat, gos, ocell\}$.
- El model prediu la probabilitat de pertinença a cada classe.

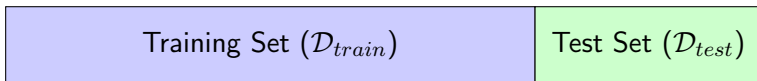
2. Regressió

- $\mathcal{Y} \subseteq \mathbb{R}$ (continu).
- Ex: Preu d'un habitatge, temperatura.
- El model prediu un valor numèric real.

El model "aprèn" uns paràmetres θ (pesos i biaixos) per tal que la predicció s'aproximi al valor real:

$$\hat{y}^{(k)} = f(\mathbf{x}^{(k)}; \theta) \approx y^{(k)}$$

Per avaluar correctament la capacitat de **generalització** (rendiment en dades futures), no podem utilitzar les mateixes dades amb les que hem ajustat el model.



- **Training Set:** Ajust de paràmetres (θ).
- **Validation Set (opcional):** Ajust d'hiperparàmetres i *early stopping*.
- **Test Set:** Avaluació final del rendiment.

Partició de Mostres: Rols Estadístics

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

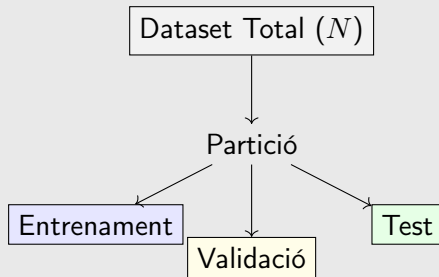
Avaluació de
Models

Generalització

Connexió
amb Xarxes

- **Mostra d'Entrenament ($\mathcal{D}_{train}$):** S'utilitza per a l'estimació de paràmetres. Minimitzem la pèrdua $L(\theta)$ sobre aquestes N_{train} mostres.
- **Mostra de Validació ($\mathcal{D}_{val}$):** S'utilitza per a la selecció del model. Com que no ha participat en l'optimització de θ , ens permet comparar diferents funcionalitats $f_1, f_2, \dots$ (o hiperparàmetres) de forma justa.
- **Mostra de Test ($\mathcal{D}_{test}$):** S'utilitza per a l'avaluació externa. És una reserva de dades que roman "invisible" durant tot el procés de disseny i selecció.

Flux de dades i independència



L'Enfocament Probabilístic del Machine Learning

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Considerem un model paramètric $f(\mathbf{x}; \theta)$. En lloc de veure'l com una simple funció de predicció, el considerem un motor que defineix una **distribució condicional**:

$$p(y \mid \mathbf{x}; \theta)$$

- El model no estima y directament, sinó els **paràmetres** d'una distribució de probabilitat.
- **Nota:** Aquest model f pot ser des d'una regressió lineal simple fins a una xarxa neuronal profunda de milions de paràmetres.

Principi de Màxima Versemblança (MLE)

Donat un conjunt de dades $\mathcal{D} = \{(\mathbf{x}^{(k)}, y^{(k)})\}_{k=1}^N$, busquem els paràmetres θ que maximitzen la probabilitat de les dades observades:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \prod_{k=1}^N p(y^{(k)} \mid \mathbf{x}^{(k)}; \theta)$$

De la Versemblança a la Negative Log-Likelihood

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Versemblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Per optimitzar computacionalment i evitar problemes de precisió numèrica, treballem amb el logaritme de la versemblança ($\mathcal{L}$):

$$\log \mathcal{L}(\theta) = \sum_{k=1}^N \log p(y^{(k)} | \mathbf{x}^{(k)}; \theta)$$

Com que els algorismes d'optimització estan fets per **minimitzar**, definim la funció de pèrdua L com:

Equivalència Fonamental: Negative Log-Likelihood (NLL)

$$L(\theta) = - \sum_{k=1}^N \log p(y^{(k)} | \mathbf{x}^{(k)}; \theta)$$

Minimitzar la Loss $L(\theta)$ és equivalent a maximitzar la Versemblança $\mathcal{L}(\theta)$.

Derivació: Regressió i MSE

Assumim que y té un soroll Gaussià al voltant de la predicció:
 $y = f(\mathbf{x}; \theta) + \epsilon$, on $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Això implica:

$$p(y | \mathbf{x}; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x}; \theta))^2}{2\sigma^2}\right)$$

Calculem la log-versemblança negativa $L(\theta)$ per a N observacions:

$$\begin{aligned} L(\theta) &= - \sum_{k=1}^N \log p(y^{(k)} | \mathbf{x}^{(k)}; \theta) \\ &= \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_{\text{constant}} + \frac{1}{2\sigma^2} \sum_{k=1}^N (y^{(k)} - f(\mathbf{x}^{(k)}; \theta))^2 \end{aligned}$$

- Si σ^2 és constant, minimitzar la NLL respecte θ equival a minimitzar:

$$\text{MSE} = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2 \implies \text{Estimador de la mitjana condicional}$$

Si assumim que l'error segueix una **distribució de Laplace**:

$$p(y \mid \mathbf{x}; \theta) = \frac{1}{2b} \exp\left(-\frac{|y - f(\mathbf{x}; \theta)|}{b}\right)$$

La log-versemblança negativa $L(\theta)$ per a N observacions independents és:

$$L(\theta) = N \log(2b) + \frac{1}{b} \sum_{k=1}^N |y^{(k)} - f(\mathbf{x}^{(k)}; \theta)|$$

L'estimador que minimitza l'error absolut mitjà (MAE) és la **mediana**.

- Si derives $\sum |y^{(k)} - \hat{y}^{(k)}|$ respecte a $\hat{y}$, obtens la suma de signes.
- El punt on la suma de signes és zero és la mediana.

Robustesa: A diferència del MSE, al MAE l'error creix linealment. Els *outliers* no desplacen l'estimador tant com ho farien amb la mitjana.

Derivació: Classificació Binària i BCE

En classificació binària, $y \in \{0, 1\}$. Modelitzem $p(y | \mathbf{x})$ com una **Bernoulli** amb paràmetre p :

$$p(y | \mathbf{x}; \theta) = p^y (1 - p)^{1-y}$$

Apliquem la NLL per a les N mostres:

$$L(\theta) = -\log \mathcal{L}(\theta) = -\sum_{k=1}^N \left[y^{(k)} \log(p_k) + (1 - y^{(k)}) \log(1 - p_k) \right]$$

- Aquesta és la **Binary Cross-Entropy (BCE)**.
- **Interpretació TI:** Mesura la divergència entre la distribució real (etiqueta) i la predita pel model.
- El model no prediu "0 o 1", sinó la probabilitat de la classe positiva ($y = 1$).
- **Conclusió:** Minimitzar la BCE maximitza la confiança en la classe correcta.
- **Nota:** Més endavant veurem que $p = \sigma(z)$, on σ és la funció sigmoide.

Representació Multiclase: One-Hot Encoding

En classificació amb K classes, no podem usar un escalar (1, 2, 3) perquè l'estadística interpretaria que la Classe 3 és "més gran" que la 1.

Convertim l'etiqueta en un vector

$\mathbf{y} \in \{0, 1\}^K$:

- **Gos:** $[1, 0, 0]^T$
- **Gat:** $[0, 1, 0]^T$
- **Ocell:** $[0, 0, 1]^T$

0
1
0

← Classe real (c)

Intuïció

El vector $\mathbf{y}$ actua com un **selector**: "encén" la posició de la classe correcta i "apaga" la resta.

De la Categòrica a la Versemblança

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Versemblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si la mostra k pertany a la classe c , tenim:

- **Target (One-Hot):** $y_j^{(k)} = 1$ si $j = c$, i 0 en cas contrari.
- **Predicció (Softmax):** $\mathbf{p}^{(k)} = [p_1^{(k)}, \dots, p_K^{(k)}]^\top$, on $\sum p_j = 1$.

Assumim una distribució **Categòrica**. La probabilitat d'observar $\mathbf{y}^{(k)}$ és:

$$p(\mathbf{y}^{(k)} | \mathbf{x}^{(k)}; \theta) = \prod_{j=1}^K (p_j^{(k)})^{y_j^{(k)}}$$

Per què aquesta fórmula?

Gràcies al One-Hot, tots els termes del producte valen 1 (perquè l'exponent és 0), **excepte el de la classe real**:

$$(p_1)^0 \cdots (p_c)^1 \cdots (p_K)^0 = p_c$$

La versemblança d'una mostra és, simplement, la probabilitat que el model assigna a la classe correcta.

Categorical Cross-Entropy (CCE)

Per optimitzar el model, minimitzem la Negative Log-Likelihood (NLL) de tot el dataset:

$$L(\theta) = - \sum_{k=1}^N \log \left(\prod_{j=1}^K (p_j^{(k)})^{y_j^{(k)}} \right) = - \sum_{k=1}^N \sum_{j=1}^K y_j^{(k)} \log(p_j^{(k)})$$

Interpretació Final

Per a cada mostra, la pèrdua és: $L^{(k)} = -\log(p_{correcte})$.

- Si el model dóna $p \approx 1$ a la classe real $\implies L \approx 0$.
- Si el model dóna $p \approx 0$ a la classe real $\implies L \rightarrow \infty$.

Minimitzar la CCE és obligar el model a posar tota la "massa de probabilitat" en la classe que indica el vector One-Hot.

Quan usar què? Motivació per al Deep Learning

Aprenentatge Automàtic II: Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Model	Tècnica Clàssica	Per què Deep Learning?
MSE	Regressió Lineal, Random Forest	Per a relacions altament no lineals i dades no estructurades (imatges, àudio).
MAE	Regressió Quantílica, Huber Regression	Quan les dades tenen molts errors de mesura o distribucions de cua pesada (<i>heavy-tailed</i>).
BCE	Regressió Logística, SVM	Per a problemes de classificació on les fronteres de decisió són extremadament complexes.
CCE	LDA, Arbres de Decisió	Per a classificació amb centenars o milers de categories (ex: reconeixement facial).

La clau: En ML clàssic, tu tries la distribució i el model és "rígid". En DL, la xarxa és un **aproximador universal** que pot aprendre paràmetres de distribucions que canvien segons x .

Resum de Pèrdues i Versemblances

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Versemblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Tasca	Supòsit $P(Y X)$	Pèrdua (NLL)	Estimador
Regressió	Gaussiana	MSE	Mitjana
Reg. Robusta	Laplace	MAE	Mediana
Clas. Binària	Bernoulli	Bin. Cross-Ent.	Probabilitat
Clas. Multi	Multinomial	Cat. Cross-Ent.	Probabilitats

Nota: Totes aquestes funcions de pèrdua s'utilitzen tant en models lineals clàssics (GLM) com en xarxes neuronals profundes.

A més de la pèrdua, avaluem resultats amb la **Matriu de Confusió**:

Real \ Pred	Pos (+)	Neg (-)
Pos (+)	TP	FN (E.II)
Neg (-)	FP (E.I)	TN

- **Err. I (FP):** Falsa alarma.
- **Err. II (FN):** Fals negatiu.

Mètriques Derivades

$$\text{Acc} = \frac{TP + TN}{N}$$

$$\text{Prec} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Nota: L'Accuracy és enganyosa en datasets desbalancejats.

Bias-Variance Tradeoff

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

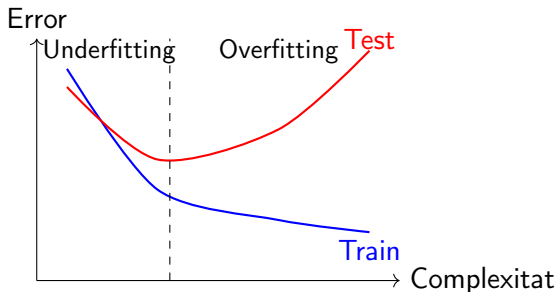
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Volem un model que funcioni bé en dades no vistes (Test), no només en Entrenament.



- **Underfitting (Subajust):** Model massa simple. Alt biaix.
- **Overfitting (Sobreajust):** Model massa complex. Alta variància.

Què és realment una Xarxa Neuronal?

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Dins el context estadístic, no és una "caixa negra".

Definició Funcional

Una xarxa neuronal és una **família paramètrica de funcions** altament flexible (no lineal) utilitzada per aproximar $f(Y | X)$.

- **Arquitectura:** Defineix la família de funcions.
- **Pesos i biaixos (θ):** Són els paràmetres a estimar.
- **Backpropagation:** Mètode numèric per maximitzar la versemblança (gradient descent).

Anem a atacar ja les Xarxes neuronals...

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

... però pas a pas!

- Entendre la **neurona artificial** com a unitat bàsica.
- Revisar el **perceptró** com a classificador lineal.
- Connectar amb la **regressió logística** (GLM, enllaç logit).
- Aplicació a **classificació** i **regressió**.
- Preparació per a **xarxes de múltiples capes (MLP)**.

Classificació

Predir etiquetes discretes $y \in \{0, 1, \dots, K\}$.

Regressió

Predir valors reals $y \in \mathbb{R}$.

Perceptró $\Rightarrow$ Regressió Logística $\Rightarrow$ Neurona Artificial

El Problema de Classificació Binària

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

- Objectiu: assignar cada mostra $x \in \mathbb{R}^d$ a $y \in \{0, 1\}$.
- Volem una frontera de decisió que separi les classes.
- El **perceptró** és el model històric bàsic.

Formulació del Perceptró

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

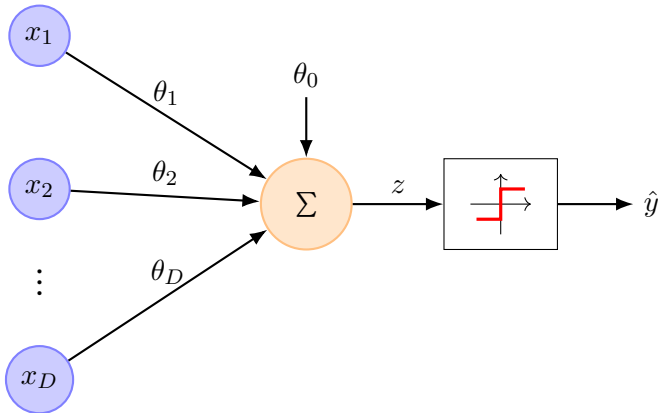
Avaluació de
Models

Generalització

Connexió
amb Xarxes

$$z = \sum_{j=1}^D \theta_j x_j + \theta_0 \quad \Rightarrow \quad \hat{y} = \begin{cases} 1 & \text{si } z \geq 0, \\ 0 & \text{si } z < 0. \end{cases}$$

- Funció d'activació: **esglaó** (Heaviside step function).
- Decisió **dura** (la sortida és binària, no una probabilitat).



Què optimitza un perceptró? (Cas $y \in \{0, 1\}$)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Volem que el nostre perceptró aprengui. Però, com sap l'algoritme si ho està fent bé o malament?

- **Intuïció:** Un punt mal classificat és una "falta". Com més lluny estigui de la seva zona correcta, més greu és l'error.
- **El problema del recompte:** Comptar quants punts fallem és una funció discreta (salts). No podem derivar-la per saber cap on moure la recta.
- **La solució:** Necessitem una funció de **Pèrdua** (L) que sigui contínua.

Què volem de la nostra Loss?

- 1 Si el punt està **ben classificat** $\Rightarrow L = 0$.
- 2 Si el punt està **mal classificat** $\Rightarrow L > 0$ (i proporcional a la distància a la frontera).

La Funció de Pèrdua del Perceptró ($y \in \{0, 1\}$)

Matemàticament, per a un conjunt de dades $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$, on cada $x^{(k)} \in \mathbb{R}^D$, definim per a cada mostra:

$$L(\theta, \theta_0) = \max(0, \underbrace{-(y^{(k)} - \hat{y}^{(k)})}_{\text{Signe de l'error}} \cdot \underbrace{(\theta^\top x^{(k)} + \theta_0)}_{\text{Distància a la frontera}})$$

Per què funciona?

- **Cas 1: Real $y^{(k)} = 1$, Predicció $\hat{y}^{(k)} = 0$.** L'error és $-(1 - 0) = -1$. La pèrdua és $-(\theta^\top x^{(k)} + \theta_0)$. Com més negatiu sigui el valor (més lluny del llindar), més gran és la pèrdua.
- **Cas 2: Real $y^{(k)} = 0$, Predicció $\hat{y}^{(k)} = 1$.** L'error és $-(0 - 1) = 1$. La pèrdua és $\theta^\top x^{(k)} + \theta_0$. Com més positiu sigui el valor, més gran és la pèrdua.

Conclusió

Minimitzar L és equivalent a "empènyer" els punts mal classificats cap a l'altra banda de la frontera.

L'actualització del Biaix i els Pesos

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

L'algoritme ajusta la frontera de decisió sumant o restant una part de l'entrada $x_j^{(k)}$ segons el signe de l'error de la mostra k .

Regla d'actualització (Gradient Descendent)

A partir de la derivada de la pèrdua anterior per a una mostra k , obtenim:

$$\theta_j \leftarrow \theta_j + \underbrace{\eta (y^{(k)} - \hat{y}^{(k)})}_{\text{error}} x_j^{(k)} \quad \forall j \in \{1, \dots, D\}$$

$$\theta_0 \leftarrow \theta_0 + \underbrace{\eta (y^{(k)} - \hat{y}^{(k)})}_{\text{error}}$$

- El **biaix** (θ_0) controla el desplaçament de la frontera sense canviar-ne l'angle.
- Els **pesos** (θ_j) controlen l'orientació (pendent) de la frontera per a cada dimensió j .

Classificació per perceptró: El conflicte del punt C

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

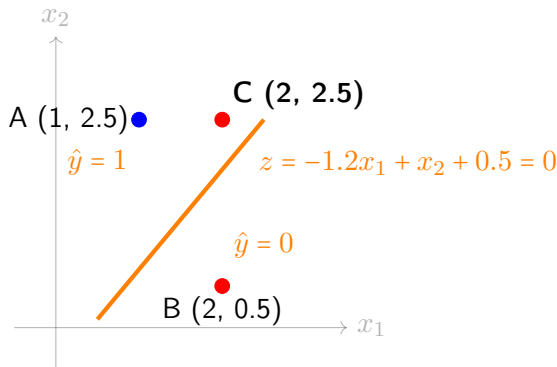
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Hem definit una frontera inicial que separa bé A i B, però s'equivoca amb C.



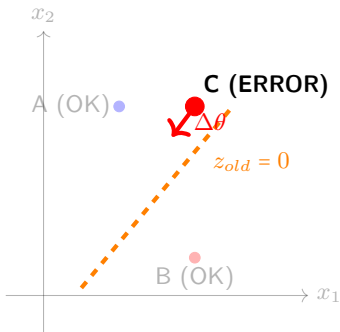
Paràmetres: $\theta = (-1.2, 1)$, $\theta_0 = 0.5$. Mirem la Loss:

- **A (1, 2.5):** $z = -1.2(1) + 2.5 + 0.5 = 1.8 > 0 \Rightarrow \hat{y} = 1$. **OK!**
- **B (2, 0.5):** $z = -1.2(2) + 0.5 + 0.5 = -1.4 < 0 \Rightarrow \hat{y} = 0$. **OK!**
- **C (2, 2.5):** $z = -1.2(2) + 2.5 + 0.5 = 0.6 > 0 \Rightarrow \hat{y} = 1$. **ERROR!**

Contribució a la Loss de C: $\ell_C = -(0 - 1) \cdot 0.6 = 0.6$.

L'ajust del model

L'algoritme utilitza la regla del Perceptró: $\Delta\theta = \eta \cdot (y - \hat{y}) \cdot x$. Només els punts amb **error** modifiquen els pesos.



1. Càlcul de l'increment ($\eta = 0.1$):

- $\Delta\theta = \eta \cdot (y^{(k)} - \hat{y}^{(k)}) \cdot \mathbf{x}^{(k)}$
- $\Delta\theta = 0.1 \cdot (-1) \cdot (2, 2.5) = (-0.2, -0.25)$
- $\Delta\theta_0 = 0.1 \cdot (-1) = -0.1$

2. Aplicació del canvi:

- $\theta_{new} = \theta_{old} + \Delta\theta$
- $\theta_{new} = (-1.2, 1) + (-0.2, -0.25) = (-1.4, 0.75)$
- $\theta_{0,new} = 0.5 + (-0.1) = 0.4$

Per què només C?

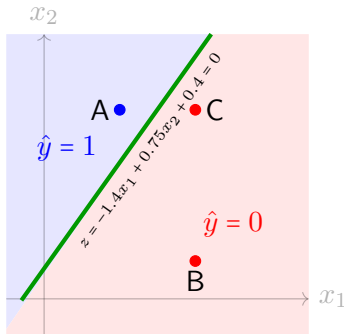
L'ajust depèn de l'error $(y^{(k)} - \hat{y}^{(k)})$:

- **A i B:** Error = 0 $\rightarrow$ Ajust = 0.
- **C:** Error = $(0 - 1) = -1$.

Intuïció

En restar una part de les coordenades de C als pesos, estem "**allunyant**" la frontera del punt C per intentar que quedi a la zona negativa.

Amb $\theta = (-1.4, 0.75)$ i $\theta_0 = 0.4$, verifiquem si hem acabat:



Verificació final:

- **A(1, 2.5):** $z = -1.4(1) + 0.75(2.5) + 0.4 = 0.875 > 0 \dots \text{OK!}$
- **B(2, 0.5):** $z = -1.4(2) + 0.75(0.5) + 0.4 = -2.025 < 0 \dots \text{OK!}$
- **C(2, 2.5):** $z = -1.4(2) + 0.75(2.5) + 0.4 = -0.525 < 0 \dots \text{OK!}$

Loss total = 0. L'algoritme ha convergit en una sola iteració.

Algorisme d'Entrenament del Perceptró

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Donada $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$, amb $y^{(k)} \in \{0, 1\}$:

- 1 Inicialitza $\theta \in \mathbb{R}^D$, $\theta_0 \in \mathbb{R}$.
- 2 Per a cada mostra $(x^{(k)}, y^{(k)})$:

$$\hat{y}^{(k)} = \mathbf{1} \left\{ \sum_{j=1}^D \theta_j x_j^{(k)} + \theta_0 \geq 0 \right\}$$

$$\begin{cases} \theta_j \leftarrow \theta_j + \eta(y^{(k)} - \hat{y}^{(k)})x_j^{(k)} & \forall j \in \{1, \dots, D\} \\ \theta_0 \leftarrow \theta_0 + \eta(y^{(k)} - \hat{y}^{(k)}) \end{cases}$$

- 3 Itera fins classificar bé (si és separable) o arribar a un màxim d'iteracions.

Nota: La codificació $y \in \{0, 1\}$ vs. $y \in \{-1, +1\}$ és equivalent via $y^{\pm 1} = 2y^{0,1} - 1$.

Teorema de Convergència del perceptró

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Siguin dos conjunts de punts en un espai D -dimensional, $\mathcal{S}_1 = \{x \in \mathbb{R}^D : y = 1\}$ i $\mathcal{S}_0 = \{x \in \mathbb{R}^D : y = 0\}$.

Definició Matemàtica de separabilitat

Els conjunts $\mathcal{S}_1$ i $\mathcal{S}_0$ són **linealment separables** si i només si existeix un vector $\theta \in \mathbb{R}^D$ i un escalar $\theta_0 \in \mathbb{R}$ tals que:

$$\forall x^{(k)} \in \mathcal{S}_1 : \theta^\top x^{(k)} + \theta_0 > 0 \quad ; \quad \forall x^{(k)} \in \mathcal{S}_0 : \theta^\top x^{(k)} + \theta_0 < 0$$

Teorema de Convergència del perceptró

Si les dades són linealment separables, l'algoritme del perceptró convergirà a una solució (Loss = 0) en un nombre **finit** d'iteracions per a qualsevol $\eta > 0$.

- **Escalat invariant:** η afecta la magnitud de $\|\theta\|$, però no l'orientació de l'hiperplà.
- **Cota de Novikoff:** El nombre màxim d'iteracions és independent de η .
- **Dinàmica:** Una η alta accelera el moviment inicial; una η baixa permet un ajust progressiu.

L'impossible: El problema XOR

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

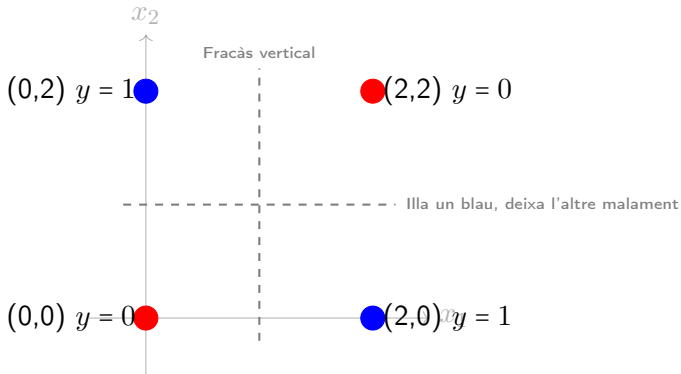
Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Intentem separar aquests 4 punts amb una sola línia recta:



Conclusió geomètrica

Cap recta pot deixar els dos punts blaus a un costat i els dos vermells a l'altre simultàniament.

Conseqüències: L'algoritme no s'atura mai

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si intentem entrenar un Perceptró amb aquestes dades:

- **Oscil·lació:** En cada pas, l'algoritme intenta corregir un error, però en fer-ho, automàticament genera un error nou en un altre punt que ja estava bé.
- **Instabilitat:** Els pesos w i el bias b canvien constantment sense arribar mai a un valor estable.
- **Manca de Convergència:** La Loss mai arriba a zero.

La gran crisi de la IA (1969)

Minsky i Papert van demostrar aquesta limitació, cosa que va provocar l'anomenat "Hivern de la IA": si un problema tan simple com el XOR no es podia resoldre, per què servirien les neurones?

Conclusió: Les limitacions del Perceptró

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Per què no podem solucionar-ho?

- Un Perceptró simple només té **una neurona**.
- Una neurona només pot generar **una frontera lineal** (una recta, un pla).

Com ho solucionem?

Necessitem "corbar-la frontera de decisió. Això s'aconsegueix:

- ➊ **Afegint capes ocultes:** Combinant diverses rectes per crear formes complexes.
- ➋ **Funcions d'activació no lineals:** Permetent que el model aprengui corbes.

"El Perceptró és un bon soldat, però per a problemes complexos necessitem un exèrcit (Xarxa Neuronal)."

L'evolució: De la frontera "dura" a la probabilitat

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

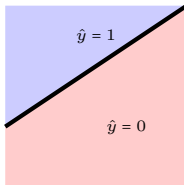
Avaluació de Models

Generalització

Connexió amb Xarxes

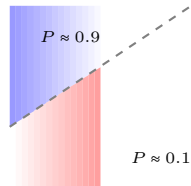
El Perceptró és un classificador sever (0 o 1).

Perceptró Simple Frontera rígidament lineal



- Sortida: **Discreta** (0, 1).
- Funció d'activació: **Step function**.
- "O ets blau o ets vermell".

Regressió Logística Frontera probabilística



- Sortida: **Contínua** (0.0 a 1.0).
- Funció d'activació: **Sigmoide**.
- "Ets blau amb probabilitat p o vermell amb probabilitat $1 - p$ ".

El salt cap a les Xarxes Neuronals

La Regressió Logística és, en realitat, **una sola neurona** amb activació sigmoide. Ens dona la base per:

- **Diferenciabilitat**: Podem usar càlcul (gradients) per aprendre millor.
- **Incertesa**: Sabem quan el model "té dubtes" (prop de la frontera).

El dilema del nom: Per què "Regressió"?

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si és per **classificar** (separar classes), per què se'n diu "regressió"?

El truc de la Regressió Logística

No estem intentant predir la classe y (que val 0 o 1) directament amb una recta, perquè una recta va de $-\infty$ a $+\infty$. El que fem és **predir (regressar) la probabilitat** que l'element pertanyi a la classe 1:

$$P(y = 1 \mid \mathbf{x}) \in [0, 1]$$

- **Problema lineal:** $z = \beta_0 + \beta_1 x_1 + \dots + \beta_D x_D$ (pot donar qualsevol valor).
- **Solució:** Hem de convertir aquest valor z perquè càpiga estrictament entre 0 i 1. Necessitem una funció de transformació o d'enllaç.

La Regressió Logística pertany a la família dels **Models Lineals Generalitzats (GLM)**. Recordem els seus 3 components:

- ❶ **Component Aleatori (Distribució):** Quina forma tenen les dades Y ?
- ❷ **Predictor Lineal:** La combinació de les variables d'entrada $z = \beta^\top x$.
- ❸ **Funció d'Enllaç (Link Function):** La funció $g(\mu)$ que connecta el valor esperat μ amb el predictor lineal z .

Com s'aplica això a la classificació binària?

- **Distribució:** $Y \sim \text{Bernoulli}(p)$. (Tirada de moneda).
- **Funció d'enllaç:** Fem servir el **Logit** (el logaritme de les *odds* o probabilitats relatives).

Desglossant la Regressió Logística com a GLM

Per definir rigorosament el model, identifiquem els **3 components del GLM**:

1. Component Aleatori (La naturalesa de les dades)

Assumim que la variable objectiu Y segueix una distribució de **Bernoulli**:

$$Y \sim \text{Bernoulli}(p) \implies P(Y = 1) = p$$

2. Predictor Lineal (L'estructura)

Construïm una combinació lineal de les variables d'entrada (igual que al Perceptró):

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_D x_D = \beta_0 + \beta^\top \mathbf{x}$$

3. Funció d'Enllaç (La connexió)

Connectem l'esperança $\mu = p$ amb el predictor lineal z mitjançant la funció **Logit**:

$$g(p) = \text{logit}(p) = \log\left(\frac{p}{1-p}\right) = z$$

*Nota: Per fer prediccions, utilitzem la inversa (l'enllaç invers), que és la **Sigmoide**: $p = \sigma(z)$.*

L'Enllaç Logit i la Sigmoide

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Dins del GLM per a la regressió logística, connectem probabilitat (p) i variables (x) així:

1. L'enllaç Logit (d'on venim):

$$\log\left(\frac{p}{1-p}\right) = z = \beta_0 + \beta_1 x_1 + \dots + \beta_D x_D$$

2. La Funció Sigmoide (l'invers): Si aïllem p d'aquesta equació, obtenim la famosa funció **Sigmoide** (σ):

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Si z és un nombre molt gran i positiu $\rightarrow p \approx 1$.
- Si z és un nombre molt negatiu $\rightarrow p \approx 0$.
- Si $z = 0 \rightarrow p = 0.5$ (Aquí hi ha la **frontera de decisió!**).

La sigmoide (Visualització)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

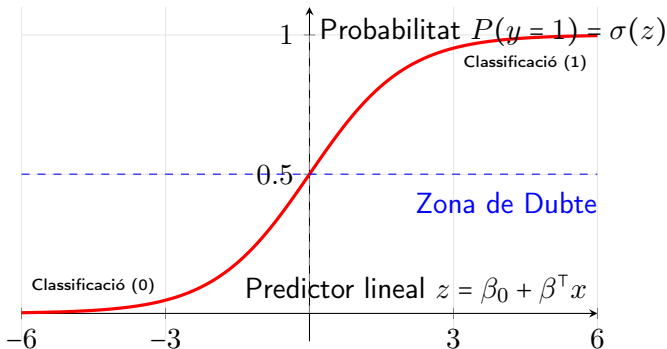
Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes



Sovint haureu vist la regressió logística escrita en termes de β :

$$z^{(k)} = \beta_0 + \beta_1 x_1^{(k)} + \dots + \beta_D x_D^{(k)} \quad z^{(k)} = \beta_0 + \beta^\top x^{(k)}$$

Canvi de notació

Ara a les β 's els hi direm θ 's. Això ens permet escriure el model de forma molt més compacta com un producte escalar:

$$z^{(k)} = \theta_0 + \theta_1 x_1^{(k)} + \dots + \theta_D x_D^{(k)} \quad \Rightarrow \quad z^{(k)} = \theta_0 + \theta^\top x^{(k)}$$

1. Versemblança (Likelihood)

Busquem els paràmetres θ i el biaix θ_0 que maximitzin la probabilitat de les dades:

$$\mathcal{L}(\theta, \theta_0) = \prod_{k=1}^N p_k^{y^{(k)}} (1 - p_k)^{1-y^{(k)}}$$

2. Funció de Pèrdua (Binary Cross-Entropy Loss)

Minimitzar la **Negative Log-Likelihood** és equivalent a maximitzar la versemblança. Definim la pèrdua L com:

$$L(\theta, \theta_0) = -\frac{1}{N} \sum_{k=1}^N \left[y^{(k)} \log(p_k) + (1 - y^{(k)}) \log(1 - p_k) \right]$$

on

$$p_k = \sigma(z^{(k)}) \quad \text{i} \quad z^{(k)} = \sum_{j=1}^D \theta_j x_j^{(k)} + \theta_0$$

Optimització (II): Derivació del Gradient

Volem minimitzar $L(\theta, \theta_0)$. Apliquem la regla de la cadena per a un pes θ_j :

$$\frac{\partial L}{\partial \theta_j} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial \theta_j}$$

$$\textcircled{1} \quad \frac{\partial L}{\partial p} = \frac{p-y}{p(1-p)}$$

$$\textcircled{2} \quad \frac{\partial p}{\partial z} = p(1-p)$$

$$\textcircled{3} \quad \frac{\partial z}{\partial \theta_j} = x_j, \quad \frac{\partial z}{\partial \theta_0} = 1$$

Equacions transcendents

Si igualem el gradient a zero per buscar el mínim analític:

$$\frac{1}{N} \sum_{k=1}^N \left(\sigma(\theta^\top x^{(k)} + \theta_0) - y^{(k)} \right) x^{(k)} = 0$$

La incògnita θ està "atrapada" dins la sigmoide. No hi ha solució tancada.

Solució: Gradient Descent

Actualitzem iterativament seguint la direcció de màxim descens:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L \quad \theta_0 \leftarrow \theta_0 - \eta \frac{\partial L}{\partial \theta_0}$$

El model ens dona $p = P(y = 1|x)$, però sovint necessitem prendre una decisió final ($\hat{y} \in \{0, 1\}$).

Regla de Decisió (Thresholding)

Definim un llindar de tall τ (habitualment 0.5):

$$\hat{y} = \begin{cases} 1 & \text{si } p \geq \tau \\ 0 & \text{si } p < \tau \end{cases}$$

Nota Pràctica: El llindar $\tau = 0.5$ no és obligatori.

- En detecció de càncer, podríem baixar τ a 0.1 per minimitzar els **Falsos Negatius** (sensibilitat alta), tot i assumir més Falsos Positius.

Limitació Important: La Linealitat Persisteix

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Tot i ser “probabilístic” i “suau”, la Regressió Logística continua sent un **Model Lineal Generalitzat**.

La frontera de decisió segueix sent una recta

Si $\tau = 0.5$, la frontera és $z = \theta_0 + \theta_1 x_1 + \dots + \theta_D x_D + = 0$. Això implica que les “isocontorns” de probabilitat són rectes paral·leles.

Conseqüència (Problema XOR): Igual que el perceptró, una sola unitat de Regressió Logística **no pot** resoldre problemes no linealment separables. Assignarà probabilitats properes a 0.5 on les classes s'encavalquen.

*Solució: Combinar múltiples unitats → **Xarxa Neuronal**.*

Visualització: El paisatge de probabilitat

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

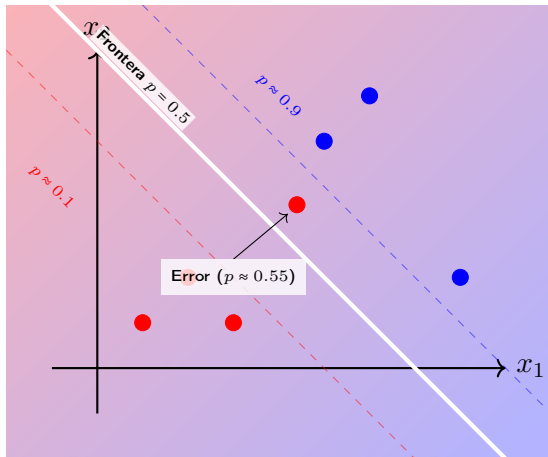
Inferència Estadística:
Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

La Regressió Logística genera una superfície de probabilitat suau.



- El color de fons indica la $P(y = 1|x)$.
- L'error prop de la frontera genera un gradient que "avisa" al model, a diferència de l'error binari del perceptró.

Matemàticament, la diferència rau en la **Funció de Pèrdua** (L) que minimitzem:

Perceptró
Hinge Loss (Discreta)

$$L \approx \max(0, -(y^{(k)} - \hat{y}^{(k)}) \cdot z^{(k)})$$

- **Penalització Dura:** Si el punt està ben classificat, la Loss és **exactament 0**.
- **Gradient:** Nul en zones correctes. Constant en zones incorrectes.
- **Conseqüència:** S'atura quan tot està separat. No busca "millorar-la separació".

Regressió Logística
Binary Cross-Entropy (Suau)

$$L = -[y^{(k)} \log(p_k) + (1 - y^{(k)}) \log(1 - p_k)]$$

- **Penalització Suau:** La Loss **mai és zero** (només asimptòticament).
- **Gradient:** Sempre actiu. El model "empeny" per allunyar els punts de la frontera.
- **Conseqüència:** Busca maximitzar el marge de confiança (probabilitat).

Una sola estructura, dues activacions

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

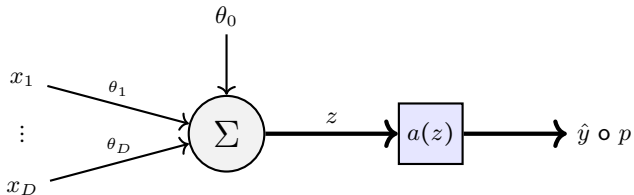
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Tant el Perceptró com la Regressió Logística comparteixen el mateix "esquelet". La diferència és només la **funció d'activació** $a(z)$.



Predictor Lineal (z)

$$z = \sum_{j=1}^D \theta_j x_j + \theta_0 \quad \text{o en forma vectorial} \quad z = \theta^\top \mathbf{x} + \theta_0$$

Extensió Multiclasse: La Funció Softmax

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quan tenim $j \in \{1, \dots, K\}$ classes, cada classe té els seus propis paràmetres:

Predictors Lineals Desdobllats

Cada classe j genera un *score* (logit) independent:

$$z_j = \theta_j^\top \mathbf{x} + \theta_{j0}$$

On θ_j és el vector de pesos per a la classe j i θ_{j0} el seu biaix.

Funció d'Enllaç (Softmax)

Transforma els scores en una distribució de probabilitat:

$$p_j = \frac{e^{z_j}}{\sum_{l=1}^K e^{z_l}}$$

Garantia: $\sum_{j=1}^K p_j = 1$ i $p_j > 0$.

Loss Function (Categorical Cross-Entropy):

$$L(\theta, \theta_0) = -\frac{1}{N} \sum_{k=1}^N \sum_{j=1}^K y_j^{(k)} \log(p_j^{(k)})$$

On $y_j^{(k)}$ és 1 si la mostra k pertany a la classe j (one-hot encoding).

Gradients: Respecte als paràmetres de la classe j :

- **Pesos:** $\nabla_{\theta_j} L = \frac{1}{N} \sum_{k=1}^N (p_j^{(k)} - y_j^{(k)}) \mathbf{x}^{(k)}$
- **Biaix:** $\frac{\partial L}{\partial \theta_{j0}} = \frac{1}{N} \sum_{k=1}^N (p_j^{(k)} - y_j^{(k)})$

Nota: L'estructura del gradient és idèntica a la regressió lineal i logística!

Limitacions: El problema de la linealitat

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

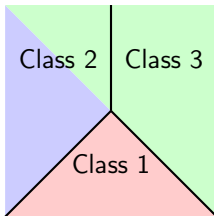
Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes



Fronteres de decisió lineals (Softmax).

Si les dades no són linealment separables (espirals, cercles), aquests models fallen.

El pas cap al Deep Learning:
Per crear fronteres corbes, hem de connectar moltes d'aquestes unitats en **capes ocultes** (hidden layers).

El Problema de Regressió

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

En lloc de predir una classe, volem predir un valor real:

$$\hat{y} = f(x^{(k)}; \theta), \quad \hat{y} \in \mathbb{R}$$

- **Exemple:** Predir el preu d'un habitatge segons la superfície i zona.
- **Objectiu:** Trobar els paràmetres θ que minimitzin la distància entre la predicció $\hat{y}^{(k)}$ i el valor real $y^{(k)}$.
- **Mètrica natural:** L'error quadràtic.

L'activació lineal = Regressió lineal

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

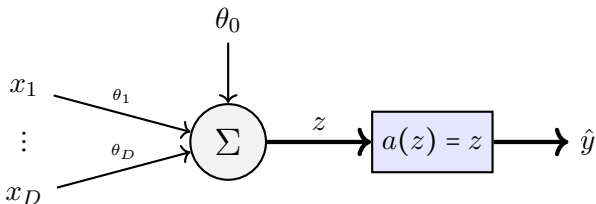
Generalització

Connexió
amb Xarxes

Si fem servir una funció d'activació **identitat** ($a(z) = z$), recuperem el model clàssic:

$$\hat{y}^{(k)} = \theta_0 + \sum_{j=1}^D \theta_j x_j^{(k)}$$

- **Interpretació:** Cada pes θ_j indica la contribució directa de la característica j al resultat final.
- **Limitació:** Només pot modelar relacions on el canvi és proporcional a l'entrada (línies planes).



Visió Unificada: El marc dels GLM

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

La Regressió Logística i la Lineal són "germanes" sota el marc dels **Models Lineals Generalitzats**. Només canvia la Loss i l'activació.

	Regressió (Lineal)	Classificació (Logística)
Sortida	$y \in \mathbb{R}$	$p \in [0, 1]$
Activació	Identitat: $\hat{y} = z$	Sigmoide: $p = \sigma(z)$
Loss (L)	MSE: $\frac{1}{N} \sum (y - \hat{y})^2$	BCE: $-\frac{1}{N} \sum y \log p + \dots$
Intuïció	Minimitzar distància	Maximitzar certesa

Conclusió per a Deep Learning

Entrenar una xarxa per regressió o classificació és gairebé idèntic: només canviem l'**última capa** i la **funció de pèrdua**.

Anatomia d'una Neurona (Unitat de processament)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

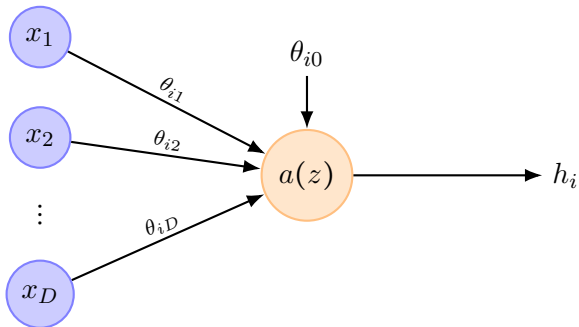
Una neurona artificial realitza dues operacions seqüencials:

- ➊ **Pre-activació (z):** Combinació lineal de les entrades (amb biaix).
- ➋ **Activació (h):** Pas a través d'una funció no lineal $a(\cdot)$.

$$z = \theta_0 + \sum_{j=1}^D \theta_j x_j, \quad h = a(z)$$

- Si $a(z)$ és lineal $\Rightarrow$ Regressió Lineal.
- Si $a(z)$ és no lineal $\Rightarrow$ Introduïm la capacitat de “corbar” l'espai i aprendre patrons complexos.

Neurona (múltiples inputs) — Cap a les capes



Notació de cara a xarxes (MLP)

Per a la neurona i d'una capa, definim:

- $\theta_i = [\theta_{i1}, \dots, \theta_{iD}]^T$ (Vector de pesos).
- θ_{i0} (Biaix individual).

Quan juntem moltes neurones, els vectors θ_i formaran les files d'una **matriu de pesos** i els θ_{i0} formaran un **vector de biaixos**.

Funcions d'activació més comunes

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

L'elecció de $a(z)$ defineix el caràcter i la capacitat d'aprenentatge de la neurona.

Sigmoide



Rang:

$[0, 1]$.

Ús: Sortida binària.

Contra: Gradient nul en zones saturades.

ReLU



Rang:

$[0, \infty)$.

Ús: Capes ocultes.

Pro: Evita el desert de gradients i és ràpida.

Identitat



Rang:

$(-\infty, \infty)$.

Ús: Regressió.

Nota: Sense no-linealitat, no hi ha "Deep" Learning.

En xarxes modernes, la ReLU és la funció d'activació més utilitzada en les capes intermèdies.

Motivació

El TensorFlow Playground permet veure com la xarxa “deforma” l'espai per separar classes.

- **Regressió Logística (0 capes, sigmoide, dataset gaussianes):**
[Enllaç configurat](#)
- **Perceptró (0 capes, activació lineal, sortida discretitzada):**
[Enllaç configurat](#)
- **Repte no lineal (circle/spiral):** activa ReLU i 2–3 capes.

Quan l'estadística lineal falla

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Victor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

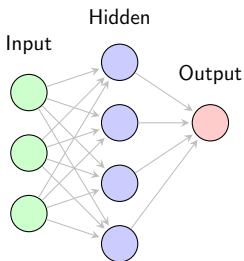
Generalització

Connexió
amb Xarxes

- Dataset **circle** o **spiral** al Playground.
- Amb 0 capes, la frontera és lineal → fracàs.
- Afegint capes i ReLU: l'espai es “doblega” i apareix una frontera no lineal.

Missatge clau

Molts perceptrons simples en capes = **aproximador universal**.



En una xarxa neuronal **totalment connectada** (FCNN), totes les neurones d'una capa reben les activacions de totes les de la capa anterior.

Una **capa** neuronal és un conjunt de neurones que operen en paral·lel.

Considerem una xarxa de K capes. La capa oculta ($l = 1$) amb $D^{(l=1)}$ neurones rep l'entrada $\mathbf{x} \in \mathbb{R}^{D^{(0)}}$. La xarxa té $\mathbb{R}^{D^{(l=K)}}$ outputs.

Denotem els índexos:

- l : índex de la capa (*layer*). La capa d'input s'etiqueta amb $l = 0$ ($l = 0, \dots, K$).
- i : índex de la neurona a la capa l ($i = 1, \dots, D^{(l)}$).
- j : índex de l'entrada als elements de la capa l ($j = 1, \dots, D^{(l-1)}$).

Nomenclatura en xarxes neuronals

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

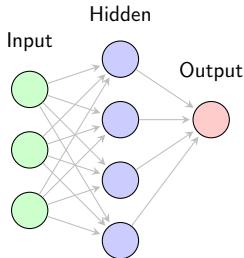
Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

- **Feed-forward network:** Les connexions de la xarxa formen un graf acíclic (sense loops).
- **Feed-forward propagation:** Procès de propagació de la informació en una feed-forward network. La sortida de la capa l es converteix en l'entrada de la capa $l + 1$.
- **Fully connected:** Apilem diverses capes de forma seqüencial i totes les sortides d'una capa van cap a tots els elements de la capa següent.



L'encadenament de capes dóna lloc al model complet:

- **Shallow networks:** Xarxes amb una única capa oculta.
- **Deep networks:** Xarxes amb múltiples capes ocultes.

Càlcul de la Pre-activació i activació a la capa oculta

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

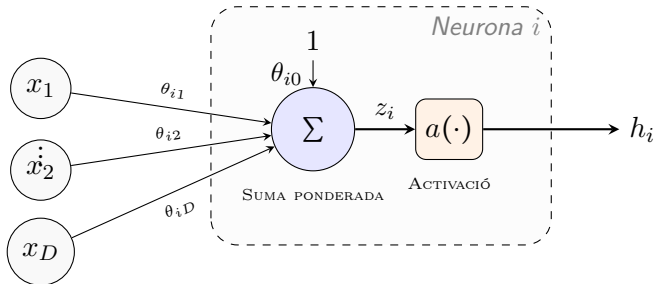
Avaluació de Models

Generalització

Connexió amb Xarxes

La pre-activació z_i de la neurona i ($i = 1, \dots, D^{(1)}$) de la capa oculta es calcula com:

$$z_i = \theta_{i0} + \sum_{j=1}^{D^{(0)}} \theta_{ij} x_j = \theta_{i0} + (\theta_{i1} \quad \theta_{i2} \quad \dots \theta_{iD^{(0)}}) \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D^{(0)} \end{pmatrix} = \theta_{i0} + \boldsymbol{\theta}_i^T \mathbf{x}$$



L'activació $h_i = a(z_i)$ és la sortida de la neurona.

Construcció de l'Estructura Matricial

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Per processar totes les $D^{(l)}$ neurones de la capa simultàniament, agrupem els seus paràmetres en:

- **Matriu de pesos Θ** amb dimensions $D^{(l)} \times D$. Cada fila i d'aquesta matriu és el vector de pesos θ_i^T de la neurona i .

$$\Theta = \begin{pmatrix} -\theta_1^T & - \\ -\theta_2^T & - \\ \vdots & \\ -\theta_{D^{(l)}}^T & - \end{pmatrix} = \begin{pmatrix} \theta_{11} & \theta_{12} & \cdots & \theta_{1D} \\ \theta_{21} & \theta_{22} & \cdots & \theta_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ \theta_{D^{(l)}1} & \theta_{D^{(l)}2} & \cdots & \theta_{D^{(l)}D} \end{pmatrix}$$

- **Vector de biaixos θ_0** (columna) de dimensió M que conté tots els biaixos individuals.

$$\theta_0 = \begin{pmatrix} \theta_{10} \\ \theta_{20} \\ \vdots \\ \theta_{D^{(l)}0} \end{pmatrix}$$

Pesos i biaixos

En una xarxa shallow ($L = 2$), identifiquem els paràmetres segons la capa de destí l :

Capa Oculta ($l = 1$): Connecta

κ_0 amb κ_1 .

- $\Theta^{(1)} \in \mathbb{R}^{D^{(1)} \times D^{(0)}}$
- $\theta_0^{(1)} \in \mathbb{R}^{D^{(1)}}$

Capa de Sortida ($l = 2$):

Connecta κ_1 amb κ_2 .

- $\Theta^{(2)} \in \mathbb{R}^{D^{(2)} \times D^{(1)}}$
- $\theta_0^{(2)} \in \mathbb{R}^{D^{(2)}}$

Definició General per a qualsevol capa l

- **Matriu de Pesos ($\Theta^{(l)}$):** Conté la força de les connexions que arriben a κ_l des de κ_{l-1} .

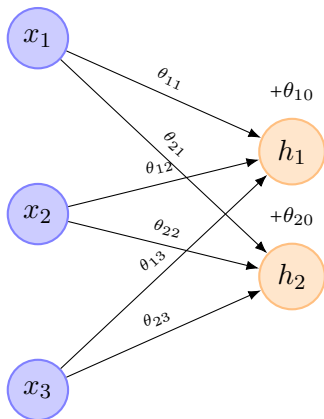
$$\Theta^{(l)} = \begin{pmatrix} \theta_{11} & \dots & \theta_{1D^{(l-1)}} \\ \vdots & \ddots & \vdots \\ \theta_{D^{(l)}1} & \dots & \theta_{D^{(l)}D^{(l-1)}} \end{pmatrix}^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l-1)}}$$

- **Vector de Biaix ($\theta_0^{(l)}$):** Un valor d'ajust propi per a cada neurona de la capa l .

$$\theta_0^{(l)} = (\theta_{10} \quad \theta_{20} \quad \dots \quad \theta_{D^{(l)}0})^T \in \mathbb{R}^{D^{(l)}}$$

Exemple d'entrada a capa oculta

Siguin $\mathbf{x} \in \mathbb{R}^3$ (entrada) i una capa amb 2 neurones:



- **Input:** $\mathbf{x} = [x_1, x_2, x_3]^T$

- **Neurona 1:**

$$z_1 = \theta_{10} + \theta_{11}x_1 + \theta_{12}x_2 + \theta_{13}x_3$$

- **Neurona 2:**

$$z_2 = \theta_{20} + \theta_{21}x_1 + \theta_{22}x_2 + \theta_{23}x_3$$

- **Matriu de pesos Θ :**

$$\Theta = \begin{pmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{pmatrix}$$

- **Vector de biaixos θ_0 :**

$$\theta_0 = \begin{pmatrix} \theta_{10} \\ \theta_{20} \end{pmatrix}$$

$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} \theta_{10} \\ \theta_{20} \end{pmatrix} = \begin{pmatrix} (\theta_{11}x_1 + \theta_{12}x_2 + \theta_{13}x_3) + \theta_{10} \\ (\theta_{21}x_1 + \theta_{22}x_2 + \theta_{23}x_3) + \theta_{20} \end{pmatrix}$$

Exemple d'operació en una xarxa 1-2-1

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

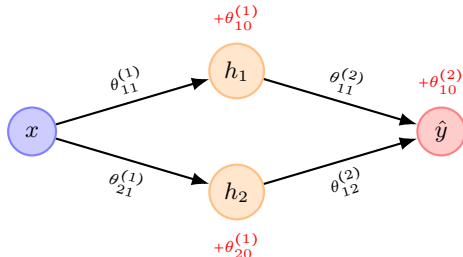
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Siguin $x \in \mathbb{R}$ (entrada) i una xarxa amb una capa oculta de 2 neurones:



Capa 1 (Oculta):

- $\Theta^{(1)} = \begin{pmatrix} \theta_{11} \\ \theta_{21} \end{pmatrix}^{(1)}$
- $\theta_0^{(1)} = \begin{pmatrix} \theta_{10} \\ \theta_{20} \end{pmatrix}^{(1)}$
- $\mathbf{z}^{(1)} = \Theta^{(1)}x + \theta_0^{(1)} = \begin{pmatrix} \theta_{11}x + \theta_{10} \\ \theta_{21}x + \theta_{20} \end{pmatrix}^{(1)}$

Capa 2 (Sortida):

- $\Theta^{(2)} = (\theta_{11} \quad \theta_{12})^{(2)}, \theta_{10}^{(2)}$
- $z^{(2)} = \Theta^{(2)}\mathbf{h}^{(1)} + \theta_{10}^{(2)}$
- $z^{(2)} = (\theta_{11}h_1 + \theta_{12}h_2)^{(2)} + \theta_{10}^{(2)}$

Paràmetres per a una xarxa 1-2-1

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

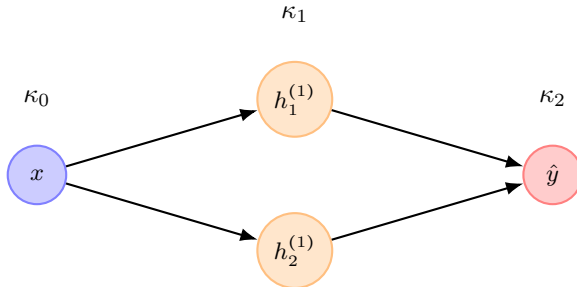
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

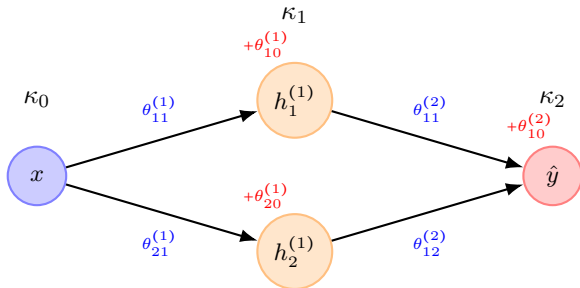
Quants paràmetres lliures té aquesta xarxa neuronal?



$$\mathcal{P} = \underbrace{D^{(1)}(D^{(0)} + 1)}_{\text{Capa } \kappa_1} + \underbrace{D^{(2)}(D^{(1)} + 1)}_{\text{Capa } \kappa_2}$$

Paràmetres per a una xarxa 1-2-1

Quants paràmetres lliures té aquesta xarxa neuronal?



$$\mathcal{P} = \underbrace{D^{(1)}(D^{(0)} + 1)}_{\text{Capa } \kappa_1} + \underbrace{D^{(2)}(D^{(1)} + 1)}_{\text{Capa } \kappa_2} \implies \mathcal{P} = \underbrace{2 \times (1 + 1)}_{4 \text{ paràmetres}} + \underbrace{1 \times (2 + 1)}_{3 \text{ paràmetres}} = 7$$

Capa κ_1 ($D^{(1)} = 2, D^{(0)} = 1$):

- 2 pesos ($\Theta^{(1)} \in \mathbb{R}^{2 \times 1}$)
- 2 biaixos ($\theta_0^{(1)}$)

Capa κ_2 ($D^{(2)} = 1, D^{(1)} = 2$):

- 2 pesos ($\Theta^{(2)} \in \mathbb{R}^{1 \times 2}$)
- 1 biaix ($\theta_0^{(2)}$)

Nombre de Paràmetres en Shallow networks

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

El nombre total de paràmetres lliures ($\mathcal{P}$) determina la complexitat i la capacitat d'aprenentatge de la xarxa. En una xarxa *fully connected*:

Fórmula General per a la Capa l

Si la capa $l - 1$ té $D^{(l-1)}$ neurones i la capa l en té $D^{(l)}$:

$$\mathcal{P}_{\text{capa } l} = \underbrace{(D^{(l)} \times D^{(l-1)})}_{\text{Pesos } \Theta^{(l)}} + \underbrace{D^{(l)}}_{\text{Biaixos } \theta_0^{(l)}} = D^{(l)}(D^{(l-1)} + 1)$$

Per a una xarxa amb una capa oculta ($D^{(0)} \rightarrow D^{(1)} \rightarrow D^{(2)}$):

$$\mathcal{P} = D^{(1)}(D^{(0)} + 1) + D^{(2)}(D^{(1)} + 1)$$

Paràmetres per a una xarxa 1-4-1

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

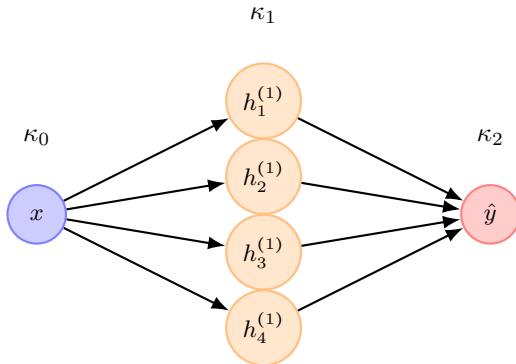
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal?



Paràmetres per a una xarxa 3-2-1

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

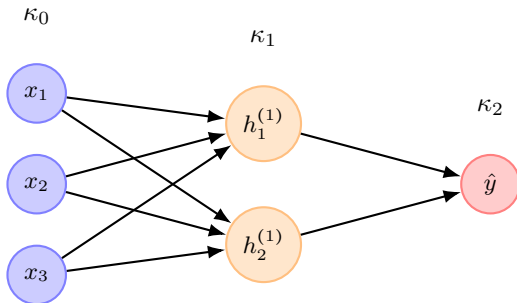
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal?



Paràmetres per a una xarxa 2-3-2

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

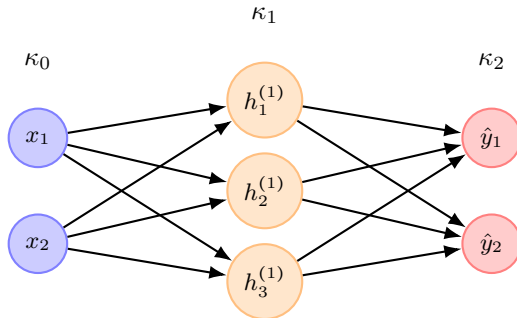
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal?



Representació Matricial (Capa l)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Per calcular les activacions de la capa l d'una vegada, operem amb la matriu de pesos $\Theta^{(l)}$ (mida $D^{(l)} \times D^{(l-1)}$) i el vector de biaixos $\theta_0^{(l)}$:

$$\underbrace{\begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_{D^{(l)}} \end{pmatrix}}_{\mathbf{z}^{(l)}} = \underbrace{\begin{pmatrix} \theta_{10} \\ \theta_{20} \\ \vdots \\ \theta_{D^{(l)}0} \end{pmatrix}}_{\theta_0^{(l)}} + \underbrace{\begin{pmatrix} \theta_{11} & \theta_{12} & \dots & \theta_{1D^{(l-1)}} \\ \theta_{21} & \theta_{22} & \dots & \theta_{2D^{(l-1)}} \\ \vdots & \vdots & \ddots & \vdots \\ \theta_{D^{(l)}1} & \theta_{D^{(l)}2} & \dots & \theta_{D^{(l)}D^{(l-1)}} \end{pmatrix}}_{\Theta^{(l)}} \underbrace{\begin{pmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_{D^{(l-1)}}^{(l-1)} \end{pmatrix}}_{\mathbf{a}^{(l-1)}}$$

D'aquesta manera, el procés de la capa queda compactat en dues passes:

- 1 **Combinació lineal:** $\mathbf{z}^{(l)} = \Theta^{(l)} \mathbf{a}^{(l-1)} + \theta_0^{(l)}$
 - 2 **Activació no-lineal:** $\mathbf{a}^{(l)} = a(\mathbf{z}^{(l)})$
- **En una Shallow Network:** Per a la capa oculta ($l = 1$), l'entrada és $\mathbf{a}^{(0)} = \mathbf{x}$.

Estructura d'una Shallow Network ($K = 2$)

Una xarxa neuronal simple (shallow) consta de dues capes de càlcul:

- **Capa d'Entrada (κ_0):** Representa les dades d'entrada $\mathbf{x} \in \mathbb{R}^{D^{(0)}}$. Definim la seva activació com:

$$\mathbf{h}^{(0)} = \mathbf{x}$$

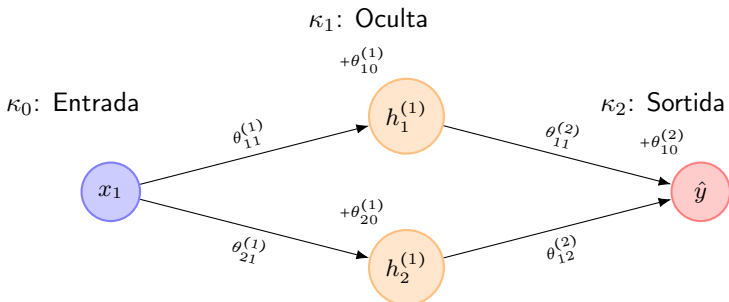
- **Capa Oculta (κ_1):** Única capa intermèdia de processament.
 - Rep com a entrada $\mathbf{h}^{(0)}$.
 - Té $D^{(1)}$ neurones, amb paràmetres $\Theta^{(1)} \in \mathbb{R}^{D^{(1)} \times D^{(0)}}$ i $\theta_0^{(1)} \in \mathbb{R}^{D^{(1)}}$.
 - El càlcul de les seves activacions és:

$$\mathbf{h}^{(1)} = a^{(1)}(\Theta^{(1)}\mathbf{h}^{(0)} + \theta_0^{(1)})$$

- **Capa de Sortida (κ_2):** Produeix la predicció final $\hat{\mathbf{y}}$.
 - Rep com a entrada les activacions de la capa oculta, $\mathbf{h}^{(1)}$.
 - Té $D^{(2)}$ sortides, amb paràmetres $\Theta^{(2)} \in \mathbb{R}^{D^{(2)} \times D^{(1)}}$ i $\theta_0^{(2)} \in \mathbb{R}^{D^{(2)}}$.
 - La predicció final es calcula com una **composició de funcions**:

$$\hat{\mathbf{y}} = \mathbf{h}^{(2)} = a^{(2)}(\Theta^{(2)}\mathbf{h}^{(1)} + \theta_0^{(2)})$$

Exemple: Flux de càlcul en xarxa 1-2-1



Exemple: Flux de càlcul en xarxa 1-2-1

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

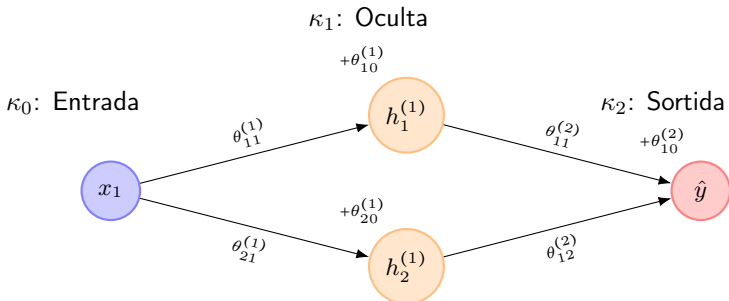
Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes



1. Pas a Capa Oculta (κ_1):

$$\Theta^{(1)} = \begin{pmatrix} \theta_{11}^{(1)} \\ \theta_{21}^{(1)} \end{pmatrix}, \quad \theta_0^{(1)} = \begin{pmatrix} \theta_{10}^{(1)} \\ \theta_{20}^{(1)} \end{pmatrix}$$

$$\mathbf{h}^{(1)} = \begin{pmatrix} h_1^{(1)} \\ h_2^{(1)} \end{pmatrix} = a^{(1)} \left(\Theta^{(1)} (x_1) + \theta_0^{(1)} \right)$$

Exemple: Flux de càlcul en xarxa 1-2-1

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

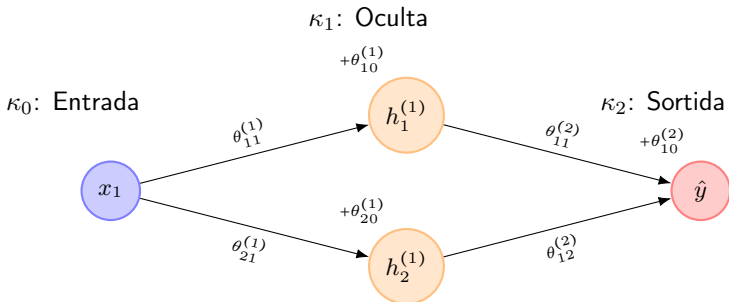
Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes



1. Pas a Capa Oculta (κ_1):

$$\Theta^{(1)} = \begin{pmatrix} \theta_{11}^{(1)} \\ \theta_{21}^{(1)} \end{pmatrix}, \quad \theta_0^{(1)} = \begin{pmatrix} \theta_{10}^{(1)} \\ \theta_{20}^{(1)} \end{pmatrix}$$

$$\mathbf{h}^{(1)} = \begin{pmatrix} h_1^{(1)} \\ h_2^{(1)} \end{pmatrix} = a^{(1)} \left(\Theta^{(1)} (x_1) + \theta_0^{(1)} \right)$$

2. Pas a Sortida (κ_2):

$$\Theta^{(2)} = \begin{pmatrix} \theta_{11}^{(2)} & \theta_{12}^{(2)} \end{pmatrix}, \quad \theta_0^{(2)} = \begin{pmatrix} \theta_{10}^{(2)} \end{pmatrix}$$

$$\begin{aligned} \hat{y} = h^{(2)} &= a^{(2)} \left(\Theta^{(2)} \mathbf{h}^{(1)} + \theta_0^{(2)} \right) \\ &= a^{(2)} \left(\theta_{11}^{(2)} h_1^{(1)} + \theta_{12}^{(2)} h_2^{(1)} + \theta_{10}^{(2)} \right) \end{aligned}$$

Exemple: Flux de càlcul en xarxa 1-2-1

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

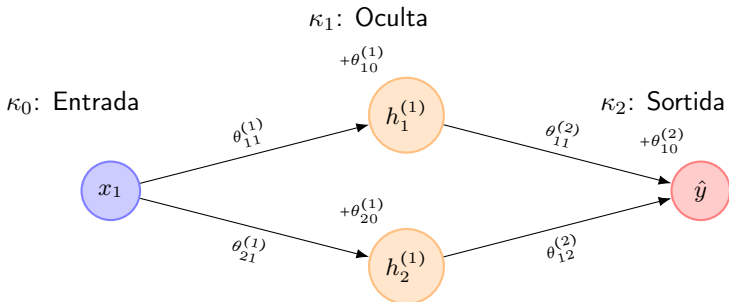
Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes



1. Pas a Capa Oculta (κ_1):

$$\Theta^{(1)} = \begin{pmatrix} \theta_{11}^{(1)} \\ \theta_{21}^{(1)} \end{pmatrix}, \quad \theta_0^{(1)} = \begin{pmatrix} \theta_{10}^{(1)} \\ \theta_{20}^{(1)} \end{pmatrix}$$

$$\mathbf{h}^{(1)} = \begin{pmatrix} h_1^{(1)} \\ h_2^{(1)} \end{pmatrix} = a^{(1)} \left(\Theta^{(1)} (x_1) + \theta_0^{(1)} \right)$$

2. Pas a Sortida (κ_2):

$$\Theta^{(2)} = \begin{pmatrix} \theta_{11}^{(2)} & \theta_{12}^{(2)} \end{pmatrix}, \quad \theta_0^{(2)} = \begin{pmatrix} \theta_{10}^{(2)} \end{pmatrix}$$

$$\begin{aligned} \hat{y} = h^{(2)} &= a^{(2)} \left(\Theta^{(2)} \mathbf{h}^{(1)} + \theta_0^{(2)} \right) \\ &= a^{(2)} (\theta_{11}^{(2)} h_1^{(1)} + \theta_{12}^{(2)} h_2^{(1)} + \theta_{10}^{(2)}) \end{aligned}$$

$$\mathbf{z}^{(1)} = \begin{pmatrix} \theta_{11} \\ \theta_{21} \end{pmatrix}^{(1)} (x) + \begin{pmatrix} \theta_{10} \\ \theta_{20} \end{pmatrix}^{(1)} \xrightarrow{a^{(1)}} \mathbf{h}^{(1)} \xrightarrow{\Theta^{(2)}} z^{(2)} = \begin{pmatrix} \theta_{11} & \theta_{12} \end{pmatrix}^{(2)} \begin{pmatrix} h_1 \\ h_2 \end{pmatrix}^{(1)} + \theta_{10}^{(2)}$$

Exemple numèric: Forward Pass (1-2-1)

Considerem una entrada $x = 2$ i el valor real $y = 10$. Per simplificar el càlcul manual, assumirem que la funció d'activació és la **identitat**: $a(z) = z$.

❶ **Capa 1** ($l = 1$): Entrem a la capa oculta.

- Pre-activació: $\mathbf{z}^{(1)} = \begin{pmatrix} 3 \\ 1 \end{pmatrix} (2) + \begin{pmatrix} 0.5 \\ -1 \end{pmatrix} = \begin{pmatrix} 6.5 \\ 1 \end{pmatrix}$
- Activació: $\mathbf{h}^{(1)} = a^{(1)}(\mathbf{z}^{(1)}) = \begin{pmatrix} 6.5 \\ 1 \end{pmatrix}$

❷ **Capa 2** ($l = 2$): Calculem la sortida final.

- Pre-activació:
 $z^{(2)} = \begin{pmatrix} 1.5 & 0.5 \end{pmatrix} \begin{pmatrix} 6.5 \\ 1 \end{pmatrix} + 0.2 = 9.75 + 0.5 + 0.2 = 10.45$
- Sortida: $\hat{y} = a^{(2)}(z^{(2)}) = 10.45$

Nota sobre $a(z)$

En un cas real, usaríem funcions com **ReLU** o **Sigmoide**. Aquí fem servir la identitat per centrar-nos en la propagació de les matrius.

Càlcul de la funció de pèrdua

Un cop tenim la predicció $\hat{y} = 10.45$ i sabem que el valor real és $y = 10$, quantifiquem l'error mitjançant la **Loss Function** $L(\hat{y}, y)$.

Si fem servir el MSE:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

En el nostre exemple:

- Diferència (residual): $10.45 - 10 = \mathbf{0.45}$
- Error quadrat: $0.45^2 = 0.2025$
- **Valor final de la Loss:** $L = \frac{1}{2}(0.2025) = \mathbf{0.10125}$

Interpretació

Aquest número (L) ens diu “com de malament” ho està fent la xarxa. L'objectiu de l'entrenament és moure els valors de Θ i θ_0 (en aquest cas, els 7 paràmetres) per tal que en la pròxima iteració L s'apropi a zero.

Exercici: Forward Pass en Regressió (3-2-1)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

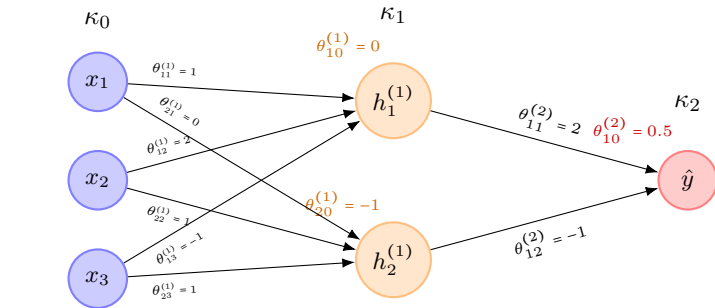
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Calculeu la sortida $\hat{y}$ per a $\mathbf{x} = (1 \ 0 \ 2)^\top$. Considereu activació **ReLU** a κ_1 ($a^{(1)}$) i **lineal** a κ_2 ($a^{(2)}$).



Classificació i la connexió amb els GLM

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

L'estructura que hem vist ($\hat{y} = a^{(2)}(\dots)$) és extremadament flexible i es comporta com un **Model Lineal Generalitzat (GLM)** on la xarxa aprèn la representació de les dades:

- **Cas Regressió:** Si volem predir un valor continu, la funció d'activació final $a^{(2)}$ acostuma a ser la **identitat**. L'error es mesura amb el MSE.
- **Cas Classificació Binària:** Si volem predir una probabilitat ($y \in \{0, 1\}$), apliquem una funció **Sigmoide** (o logística) a l'última capa:

$$\hat{y} = a(z^{(2)}) = \frac{1}{1 + e^{-z^{(2)}}}$$

Interpretació com a GLM

Una xarxa neuronal es pot veure com una regressió logística (o lineal) on, en lloc d'usar les entrades x originals, fem servir les activacions de l'última capa oculta $\mathbf{h}^{(K-1)}$:

$$\hat{y} = \text{link}^{-1} \left(\Theta^{(K)} \mathbf{h}^{(K-1)} + \theta_0^{(K)} \right)$$

La xarxa no només calcula el model, sinó que **aprèn de forma automàtica la funció d'enllaç (*link function*)** i la transformació de les dades.

Classificació: Què fa realment la xarxa?

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Davant d'un problema no linealment separable (com el XOR), la xarxa actua en dues etapes:

- ➊ **Transformació de l'espai (Capes Ocultes):** Les funcions d'activació no-lineals (Step, σ , ReLU) "dobleguen" o "estiren" l'espai d'entrada. Els punts es desplacen a una nova configuració.
- ➋ **Separació Lineal (Capa de Sortida):** En aquest nou espai transformat, la capa de sortida traça un **hiperplà recte** per separar les classes.

Conclusió

La "frontera corba" que veiem en l'espai original és la projecció d'un tall recte en un espai que ha estat prèviament deformat per la xarxa.

L'activació és el que permet "doblegar" el paper. Sense activació, per moltes capes que tinguéssim, el paper sempre seria pla i no podríem resoldre el XOR.

El problema XOR: Recordatori visual

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

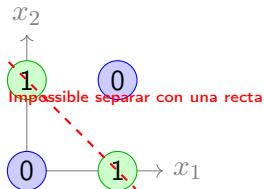
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Tenim quatre punts en l'espai $\mathbb{R}^2$. Volem que la xarxa doni un 1 quan les entrades són diferents i un 0 quan són iguals.

x_1	x_2	y (Sortida)
0	0	0
0	1	1
1	0	1
1	1	0



- Un perceptró defineix un semi-espai lineal: $h = a(\theta_1 x_1 + \theta_2 x_2 + \theta_0)$.
- $a()$ és una funció esglaó (*step*).
- Per resoldre el XOR, necessitem **dues** línies que treballin juntes.

Arquitectura de la solució: Capa oculta

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

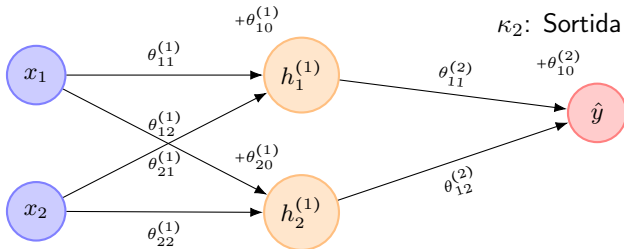
Connexió amb Xarxes

Proposem una xarxa amb 2 entrades, 2 neurones perceptró ocultes i 1 sortida amb activació perceptró.

κ_0 : Entrada

κ_1 : Oculta

κ_2 : Sortida



Pas 1 (κ_1):

$$\mathbf{h}^{(1)} = a^{(1)} \left(\Theta^{(1)} \mathbf{x} + \theta_0^{(1)} \right)$$

On $\Theta^{(1)}$ és 2×2 . Cada neurona $h_i^{(1)}$ crearà una frontera diferent.

Pas 2 (κ_2):

$$\hat{y} = a^{(2)} \left(\Theta^{(2)} \mathbf{h}^{(1)} + \theta_{10}^{(2)} \right)$$

κ_2 combinarà els resultats de les dues fronteres anteriors.

Solució XOR: Geometria de la Capa Oculta

Capa Oculta (κ_1):

$$\Theta^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \theta_0^{(1)} = \begin{pmatrix} -0.5 \\ -1.5 \end{pmatrix}$$

Les neurones de κ_1 defineixen dos semiplans en $\mathbb{R}^2$:

Neurona $h_1^{(1)}$ (Semipla H_1):

$$z_1 = x_1 + x_2 - 0.5 \geq 0$$

$$H_1 = \{(x_1, x_2) \in \mathbb{R}^2 : x_2 \geq -x_1 + 0.5\}$$

Neurona $h_2^{(1)}$ (Semipla H_2):

$$z_2 = x_1 + x_2 - 1.5 \geq 0$$

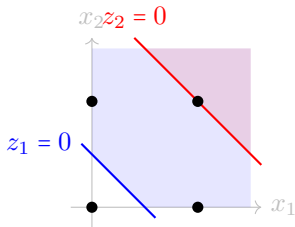
$$H_2 = \{(x_1, x_2) \in \mathbb{R}^2 : x_2 \geq -x_1 + 1.5\}$$

Observem que $H_2 \subset H_1$. Els punts d'interès són:

- $(0, 1), (1, 0) \in H_1 \setminus H_2$ (Dins d'un, fora de l'altre)
- $(1, 1) \in H_1 \cap H_2$ (Dins de tots dos)

Capa de Sortida (κ_2):

$$\Theta^{(2)} = \begin{pmatrix} 1 & -2 \end{pmatrix}, \quad \theta_{10}^{(2)} = -0.5$$



Resolent el XOR amb el Perceptró Multicapa

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

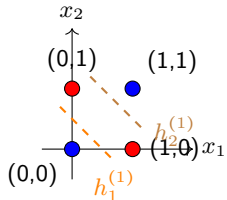
Connexió
amb Xarxes

Una xarxa 2-2-1 de perceptrons ($\text{step}(z)$) resol el XOR mitjançant una transformació de l'espai:

1. Capa Oculta (2 perceptrons):

Cada neurona crea una frontera lineal:

- $h_1^{(1)} = \text{step}(x_1 + x_2 - 0.5)$ Porta OR: S'activa si hi ha almenys un 1.
- $h_2^{(1)} = \text{step}(x_1 + x_2 - 1.5)$ Porta AND: Només s'activa si tots dos són 1.



2. Capa de Sortida: Combina les respostes per aïllar el XOR (OR - AND):

$$\hat{y} = \text{step}(h_1^{(1)} - h_2^{(1)} - 0.5)$$

Interpretació Geomètrica

La capa oculta transforma els 4 punts originals en un nou espai (h_1, h_2) on les classes ja són **linealment separables**.

Solució XOR: Separabilitat en l'espai h

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

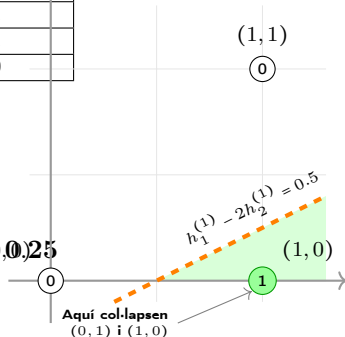
La capa κ_1 projecta els punts originals $\mathbf{x} \in \mathbb{R}^2$ als vèrtexs de l'hipercub $\{0, 1\}^2$:

$\mathbf{x}$	$\mathbf{h} = (h_1^{(1)}, h_2^{(1)})$	$z_{out} = 1h_1^{(1)} - 2h_2^{(1)} - 0.5$
(0, 0)	(0, 0)	$-0.5 < 0 \implies \hat{y} = 0$
(0, 1)	(1, 0)	$+0.5 \geq 0 \implies \hat{y} = 1$
(1, 0)	(1, 0)	$+0.5 \geq 0 \implies \hat{y} = 1$
(1, 1)	(1, 1)	$-1.5 < 0 \implies \hat{y} = 0$

L'hiperplà de decisió (κ_2): La frontera de decisió a la sortida és la recta:

$$h_1^{(1)} - 2h_2^{(1)} - 0.5 = 0 \iff h_2^{(1)} = 0.5h_1^{(1)} - 0.25$$

Aquesta recta separa el clúster del "1" dels punts "0".



Conclusió Fonamental

La capa oculta realitza un **mapeig no lineal** que col·lapsa els punts del XOR en un espai on sí que existeix una solució lineal.

Funcions d'Activació: La Unitat ReLU

Aprenentatge Automàtic II: Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

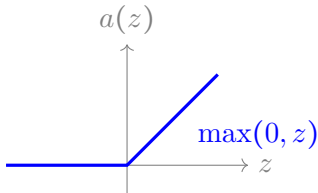
Generalització

Connexió amb Xarxes

Una de les funcions d'activació més comunes és la **Unitat Lineal Rectificada** (*Rectified Linear Unit* o **ReLU**):

$$a(z) = \max(0, z)$$

- A la capa κ_1 , cada neurona calcula: $h_i^{(1)} = a(\theta_{i1}^{(1)} x + \theta_{i0}^{(1)})$.
- El biaix $\theta_{i0}^{(1)}$ desplaça el plec horitzontalment.
- El pes $\theta_{i1}^{(1)}$ controla el pendent de la part activa.
- La combinació lineal a κ_2 permet sumar aquests plecs per "dibuixar-la funció desitjada.



Exemple d'Arquitectura 1-3-1

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

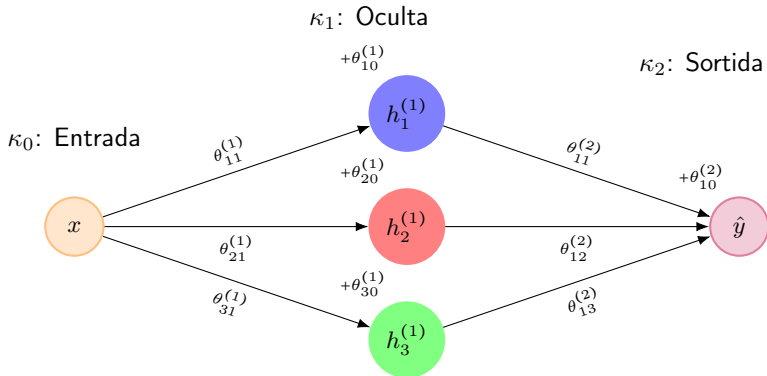
Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Considerem una xarxa poc profunda amb 1 entrada, 3 neurones ocultes i 1 sortida:



Capa oculta (κ_1):

$$\mathbf{h}^{(1)} = a(\Theta^{(1)}x + \theta_0^{(1)})$$

Amb $\Theta^{(1)} \in \mathbb{R}^{3 \times 1}$

Capa sortida (κ_2):

$$\hat{y} = a(\Theta^{(2)}\mathbf{h}^{(1)} + \theta_{10}^{(2)})$$

Amb $\Theta^{(2)} \in \mathbb{R}^{1 \times 3}$

Què fa cada neurona amb ReLU?

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

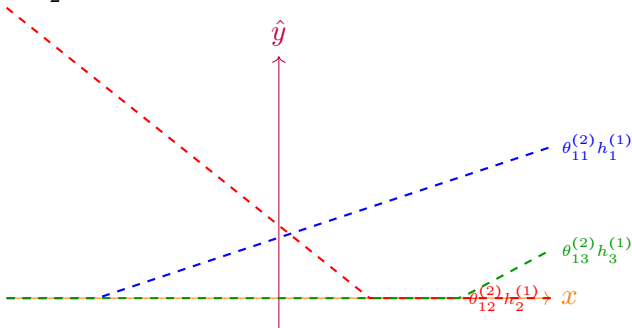
Inferència Estadística: Pèrdua i Variança

Avaluació de Models

Generalització

Connexió amb Xarxes

La funció final $\hat{y}$ es construeix sumant les contribucions de cada neurona ReLU de la capa oculta κ_1 , ponderades pels pesos de la capa κ_2 .



- Cada neurona $h_i^{(1)}$ "s'activa" en un punt diferent $x_{p_i} = -\theta_{i0}^{(1)} / \theta_{i1}^{(1)}$.
- En cada x_{p_i} , el pendent de la funció canvia, creant un nou **plec**.
- La combinació de $D^{(1)}$ neurones permet aproximar funcions no lineals.

Què fa cada neurona amb ReLU?

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

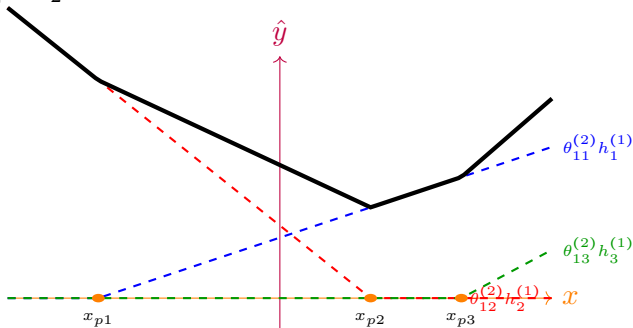
Inferència Estadística: Pèrdua i Variança

Avaluació de Models

Generalització

Connexió amb Xarxes

La funció final $\hat{y}$ es construeix sumant les contribucions de cada neurona ReLU de la capa oculta κ_1 , ponderades pels pesos de la capa κ_2 .



- Cada neurona $h_i^{(1)}$ "s'activa" en un punt diferent $x_{p_i} = -\theta_{i0}^{(1)} / \theta_{i1}^{(1)}$.
- En cada x_{p_i} , el pendent de la funció canvia, creant un nou **plec**.
- La combinació de $D^{(1)}$ neurones permet aproximar funcions no lineals.

Visualització: Creació de Plecs

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

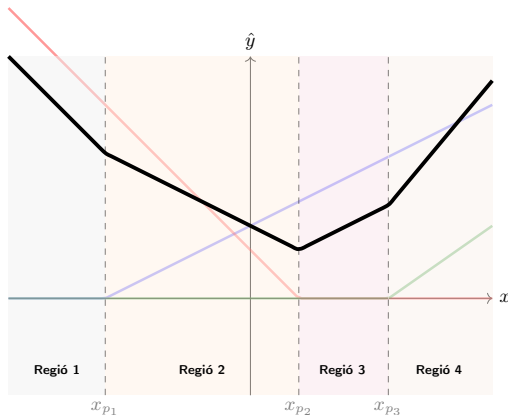
Inferència Estadística:
Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Cada neurona oculta introdueix un punt d'inflexió (x_{p_i}), dividint l'espai en regions lineals.



La combinació de 3 neurones ReLU crea una funció amb 3 plecs i 4 regions lineals. A mesura que afegim neurones, podem aproximar corbes cada cop més complexes.

Una xarxa amb activació ReLU és una funció **lineal a trossos**. Les regions lineals són una **partició de l'espai d'entrada** ($\mathbb{R}^{D^{(0)}}$).

Nombre de regions lineals (Zaslavsky, 1975)

Si tenim una capa oculta amb $D^{(1)}$ neurones i una entrada de dimensió $D^{(0)}$ (amb $D^{(1)} > D^{(0)}$), el nombre màxim de regions lineals $\mathcal{R}$ que la xarxa pot identificar és:

$$\mathcal{R} \leq \sum_{j=0}^{D^{(0)}} \binom{D^{(1)}}{j}$$

- **Exemple:** Per a la xarxa 1-3-1 anterior ($D^{(0)} = 1$) amb 3 neurones ocultes:

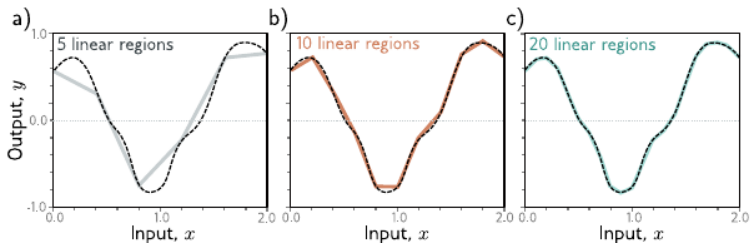
$$\mathcal{R} = \binom{3}{0} + \binom{3}{1} = 1 + 3 = 4 \text{ regions}$$

Afegim més neurones a la capa oculta?

- **Creixement Polinòmic amb l'amplada de la capa:** Si fixem la dimensió d'entrada $D^{(0)}$ i augmentem les neurones $D^{(1)}$, el nombre de regions creix com $\mathcal{R} \sim O((D^{(1)})^{D^{(0)}})$.
- **El problema de la dimensió:** Si l'entrada és gran (ex. MNIST, $D^{(0)} = 784$), el sumatori binomial de $\mathcal{R}$ creix molt lentament respecte als paràmetres que afegim.

Independència de la sortida

El nombre de regions $\mathcal{R}$ **no depèn de la dimensió de l'output** (D^{out}). Si augmentem els outputs, tindrem més funcions, però totes compartiran la mateixa malla de regions al domini d'entrada.



Nombre de regions vs. neurones i paràmetres

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

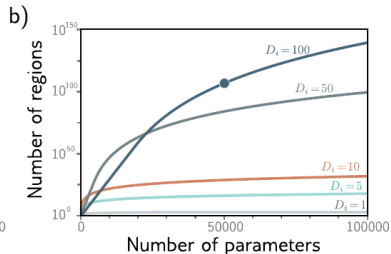
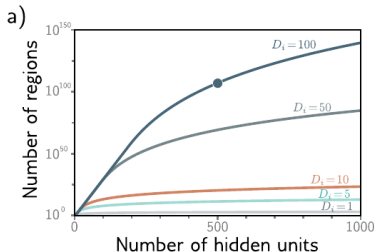
Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes



Nota: $D_i = D^{(0)}$ correspòn a la dimensió dels inputs ($\mathbf{x} \in \mathbb{R}^{D^{(0)}}$) a la capa d'entrada κ_0 .

Teorema d'Aproximació Universal (UA) I

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

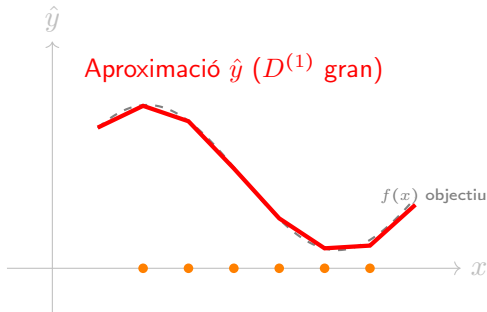
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Enunciat informal: Una xarxa neuronal amb una única capa oculta (κ_1) pot aproximar qualsevol funció contínua amb una precisió arbitrària, sempre que tingui un nombre suficient de neurones.



- Cada neurona a κ_1 afegeix un nou segment lineal (un plec).
- Si el nombre de neurones $D^{(1)} \rightarrow \infty$, els segments es fan tan petits que podem resseguir qualsevol corba.
- **Conclusió:** Les xarxes neuronals són *universal function approximators*.

I si canvio l'activació? Xarxa de Perceptrons

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

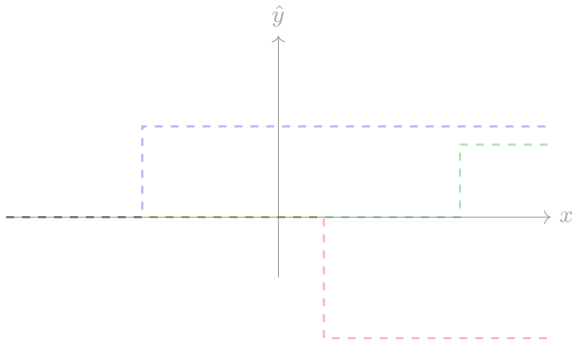
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si substituïm la ReLU per la funció escaló (*step*), la xarxa (ara de perceptrons) 1-3-1 reconstrueix la funció mitjançant la suma de funcions constants a trossos (discontínues).



- El salt de cada neurona ocorre a: $x_{Pi} = -\frac{\theta_{i0}^{(1)}}{\theta_{i1}^{(1)}}$.
- Cada pes $\theta_{1i}^{(2)}$ de la capa de sortida determina la **magnitud i el sentit** del salt (si $\theta < 0$, l'esglaó baixa).
- Amb moltes neurones, podem aproximar una funció contínua per "rectangles" (Aproximació de Riemann).

I si canvio l'activació? Xarxa de Perceptrons

Aprenentatge Automàtic II:
Fonaments de l'Apre-
nentatge Automàtic i
Deep Learning

Víctor Navas Portella

Context: AI,
ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

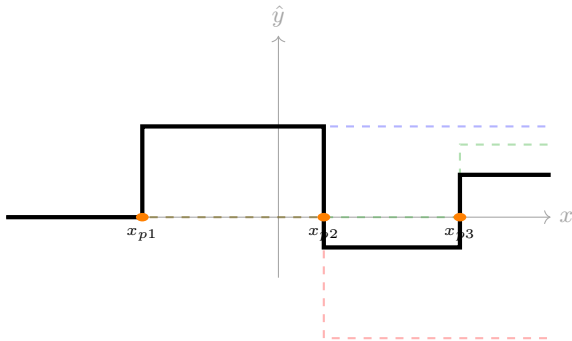
Inferència Estadística:
Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Si substituïm la ReLU per la funció escaló (*step*), la xarxa (ara de perceptrons) 1-3-1 reconstrueix la funció mitjançant la suma de funcions constants a trossos (discontínues).



- El salt de cada neurona ocorre a: $x_{p_i} = -\frac{\theta_{i0}^{(1)}}{\theta_{i1}^{(1)}}$.
- Cada pes $\theta_{1i}^{(2)}$ de la capa de sortida determina la **magnitud i el sentit** del salt (si $\theta < 0$, l'esglao baixa).
- Amb moltes neurones, podem aproximar una funció contínua per "rectangles" (Aproximació de Riemann).

Sortides Multidimensionals

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si la xarxa té $D^{(k)} > 1$ sortides, ja no aproximem una "corba", sinó que definim una **hipersuperfície** a $\mathbb{R}^{D^{(k)}}$. La xarxa aprèn a deformar i reconstruir geometries complexes.

- Cada neurona de sortida $\hat{y}_i$ representa una **coordenada** en l'espai de destí $\mathbb{R}^{D^{(K)}}$.
- El conjunt de totes les sortides defineix una **hipersuperfície** (o varietat) en l'espai $\mathbb{R}^{D^{(K)}}$.
- Les capes ocultes actuen com un sistema de **coordenades curvilínies** que s'adapten a la curvatura de les dades.

Exemples concrets de sortida $\hat{\mathbf{y}} \in \mathbb{R}^{D^{(K)}}$

- **Generació d'imatges:** La xarxa rep un codi latent (vector petit) i la sortida són els milers de píxels d'una imatge. Cada neurona de sortida determina la intensitat d'un píxel $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$.
- **Segmentació Semàntica:** Per a cada píxel d'una imatge d'entrada, la xarxa prediu a quina categoria pertany (cotxe, cel, carretera).
- **Detecció d'objectes (Bounding Boxes):** La xarxa prediu simultàniament quatre valors: centre (x, y) , amplada i alçada de la caixa que envolta un objecte.

Generalització: Cas 1-3- N

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

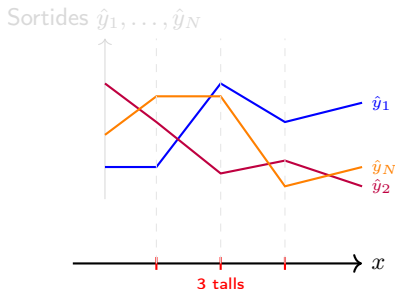
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Què passa si mantenim 3 neurones ocultes però augmentem la sortida a N dimensions?

- **Domini** ($x \in \mathbb{R}^1$): Segueix dividit en **4 regions** lineals per les 3 ReLUs.
- **Codomini** ($\hat{y} \in \mathbb{R}^N$): Cada regió k defineix una transformació lineal específica:



Conclusió

A cada regió tenim un **vector de valors** $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N]^T$. El nombre de "trossos" (regions) és una propietat estructural de la capa oculta, no de la dimensió de la sortida.

Teorema d'Aproximació Universal (UA) II

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Història i Formalització:

- Demostrat originalment per **George Cybenko** (1989) per a funcions sigmoïdes.
- Ampliat per **Kurt Hornik** (1991) demostrant que l'arquitectura és el que dóna el poder, no la funció específica.

Condicions Necessàries:

- ❶ **Capa oculta suficientment ampla:** El teorema garanteix que existeix un nombre de neurones $D^{(1)}$ que assoleix l'error desitjat.
- ❷ **Activació No Lineal:** La funció $a(z)$ ha de ser *no-polinòmica*. Si fos lineal, la xarxa col·lapsaria en una simple regressió lineal.

Què pot aproximar?

- Qualsevol funció contínua en subconjunts compactes de $\mathbb{R}^n$.
- *La "lletra petita":* El teorema diu que l'aproximació **existeix**, però no diu com trobar els pesos Θ òptims ni garanteix que el nombre de neurones necessari sigui pràctic (podria ser de milions).

Espai de funcions contínues

○ Xarxa amb $D^{(1)}$ neurones

Teorema d'Aproximació Universal (UA) III

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

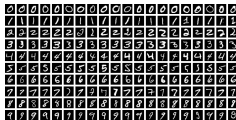
Theorem (Cybenko, 1989; Hornik, 1991)

*Sigui f una funció contínua definida en un conjunt compacte $\mathcal{C} \subset \mathbb{R}^{D^{(0)}}$.
Sigui $a(\cdot)$ una funció d'activació no constant, acotada i contínua.
Per a qualsevol $\epsilon > 0$, existeix una xarxa neuronal amb una única capa
oculta (κ_1) amb $D^{(1)}$ neurones tal que la seva sortida $\hat{y}(x)$ satisfà:*

$$\max_{x \in \mathcal{C}} |f(x) - \hat{y}(x)| < \epsilon$$

- **La no-linealitat com a pilar:** L'única condició sobre $a(\cdot)$ és que sigui **no polinòmica**. Això inclou la funció escaló (perceptró), sigmoides i ReLU.
- **Existència vs. Constructibilitat:** El teorema garanteix que la configuració de pesos $\{\Theta^{(l)}, \theta_0^{(l)}\}$ **existeix**, però no proporciona un mètode per trobar-los (optimització).
- **Capacitat:** Una xarxa "poc profunda" (*shallow*) és suficientment potent, tot i que en la pràctica les xarxes "profundes" (*deep*) són més eficients en paràmetres.

MNIST: El cost de la ineficiència Shallow



Classificació d'imatges $D^{(0)} = 28 \times 28 = 784$ en nombres del 0 al 9.

- **Complexitat (Regions):** $\mathcal{R} = \sum_{j=0}^{784} \binom{D^{(1)}}{j}$. Si $D^{(1)} \gg 784$, el terme dominant és $\binom{D^{(1)}}{784} \approx \frac{(D^{(1)})^{784}}{784!}$.
- **Cost (Paràmetres):**
 $\mathcal{P} = D^{(1)} \cdot (D^{(0)} + 1) + D^{(K=2)}(D^{(1)} + 1) \approx 785 \cdot D^{(1)}.$

- El nombre de paràmetres $\mathcal{P}$ creix de forma **lineal** amb $D^{(1)}$.
- El nombre de regions $\mathcal{R}$ creix de forma **polinòmica** de grau 784.

Encara que el grau polinòmic en $\mathcal{R}$ és alt, per a valors realistes de $D^{(1)}$ (1.000 – 5.000), no creix prou ràpid com per caputrar la variabilitat del dataset. Estem invertint milions de paràmetres per una complexitat que no escala prou ràpid.

Com podem fer que $\mathcal{R}$ creixi més ràpid?

Concatenem xarxes shallow

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

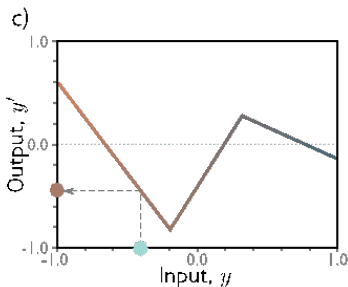
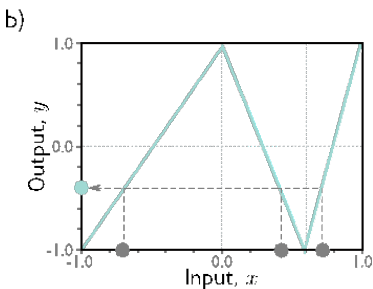
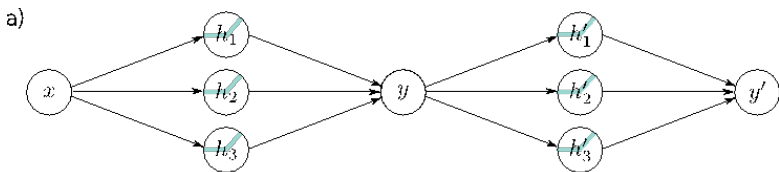
Inferència Estadística:
Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

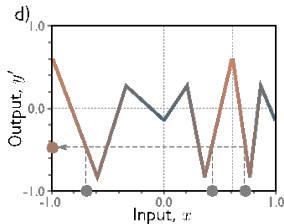
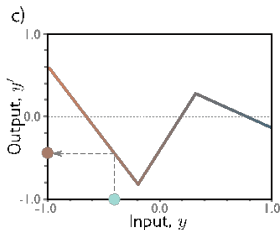
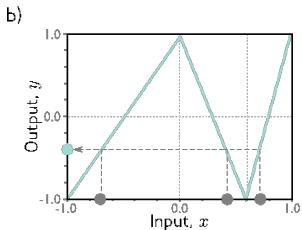
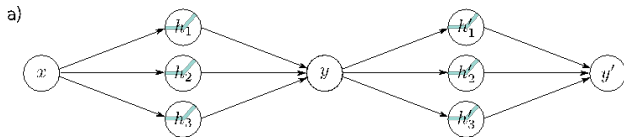
Inferència Estadística:
Pèrdua i Versemblança

Avaluació de Models

Generalització

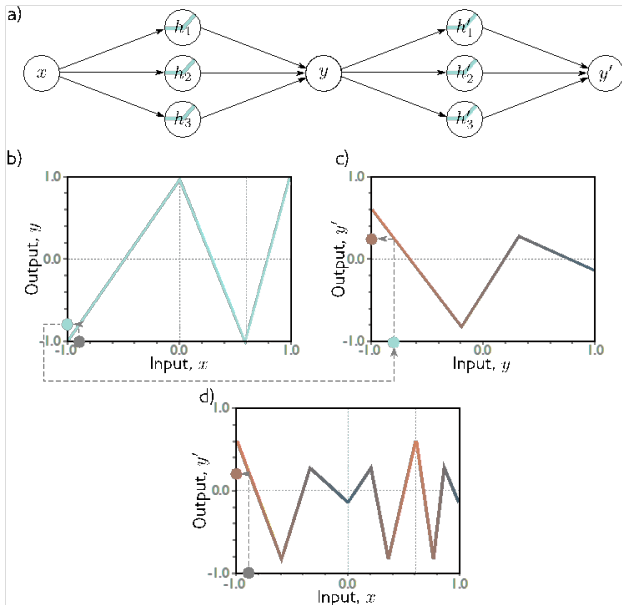
Connexió amb Xarxes

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

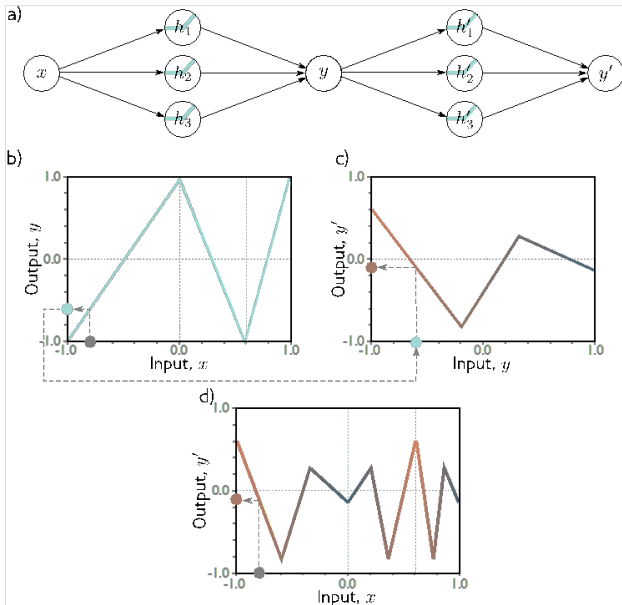
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

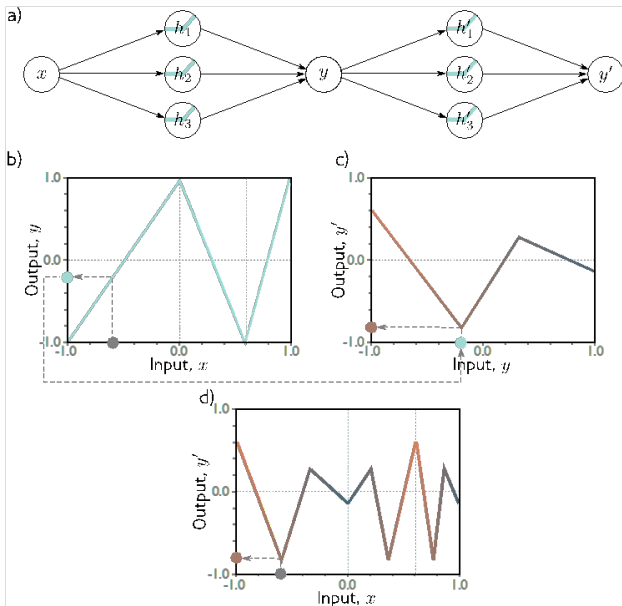
Avaluació de
Models

Generalització

Connexió
amb Xarxes

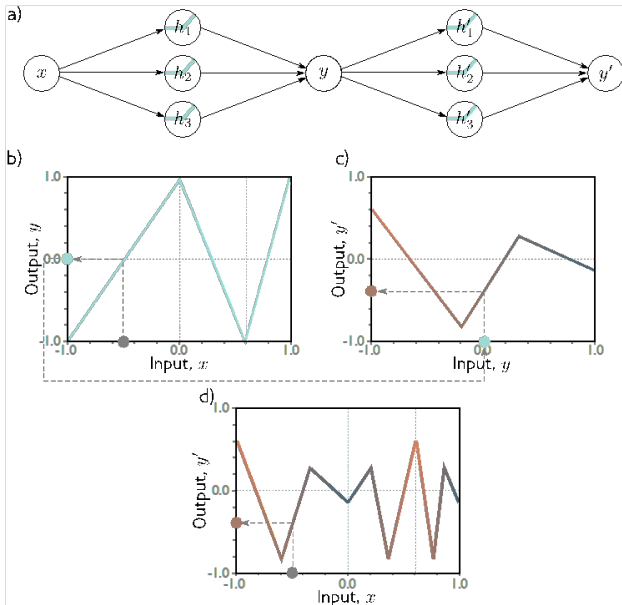
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



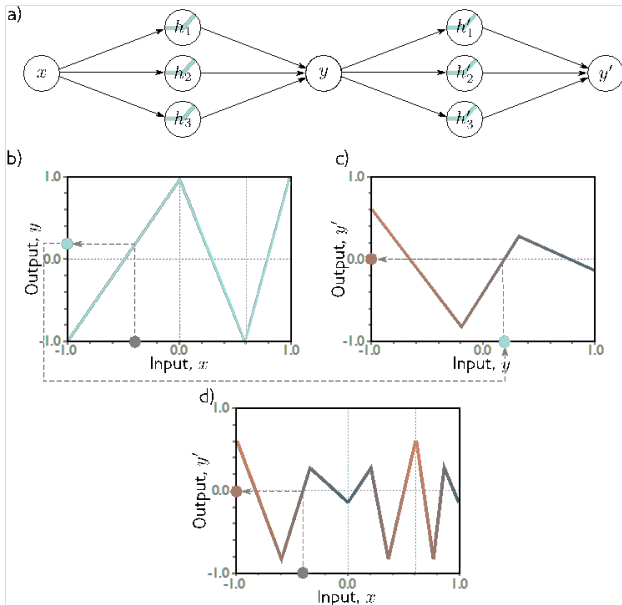
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

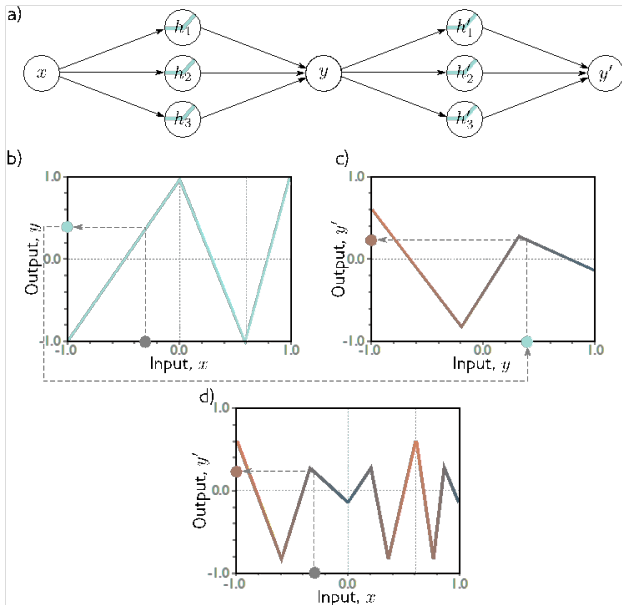
Avaluació de
Models

Generalització

Connexió
amb Xarxes

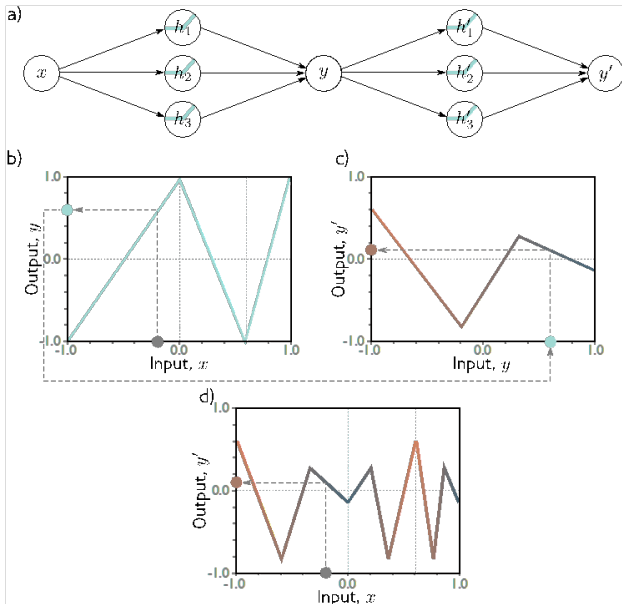
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



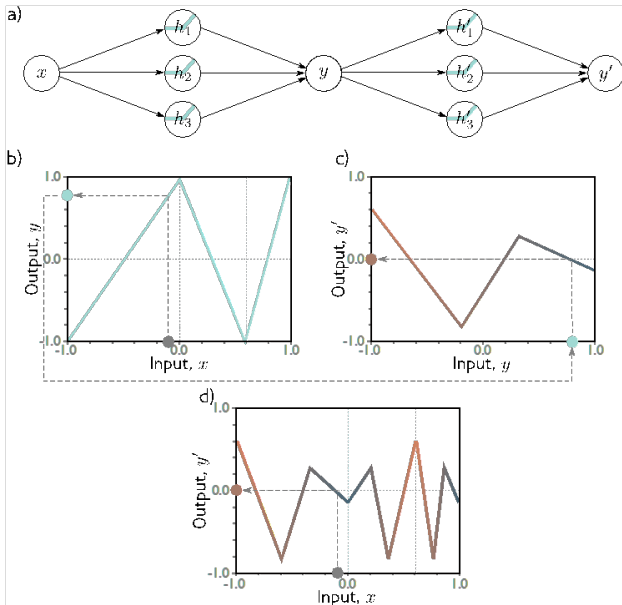
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

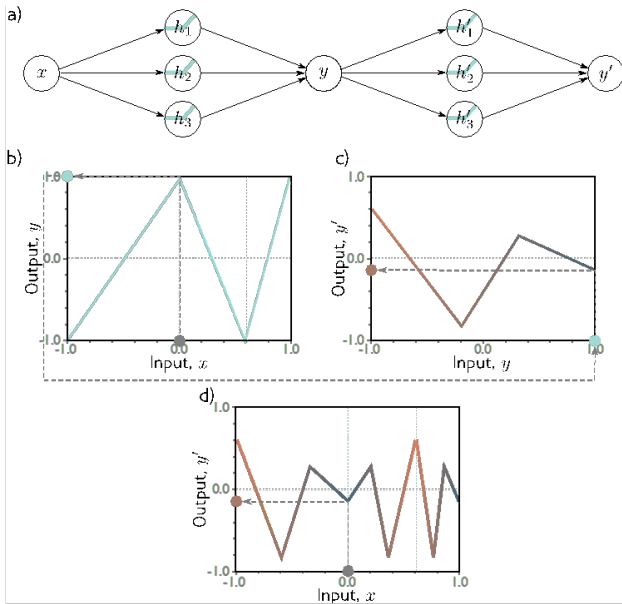
Avaluació de
Models

Generalització

Connexió
amb Xarxes

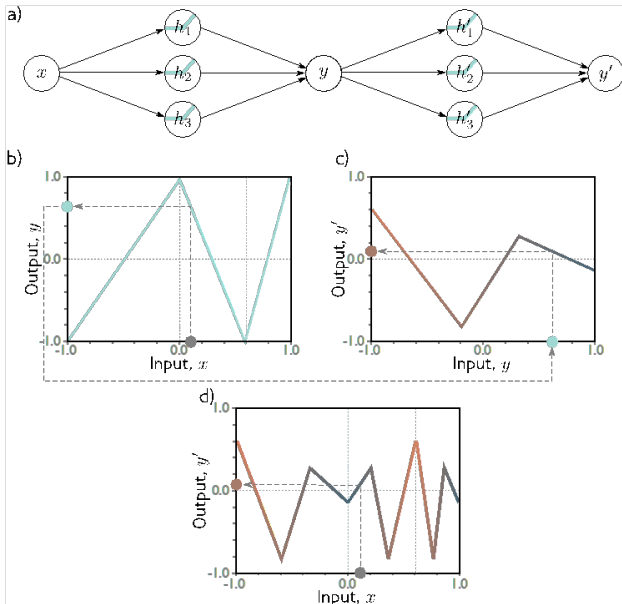
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



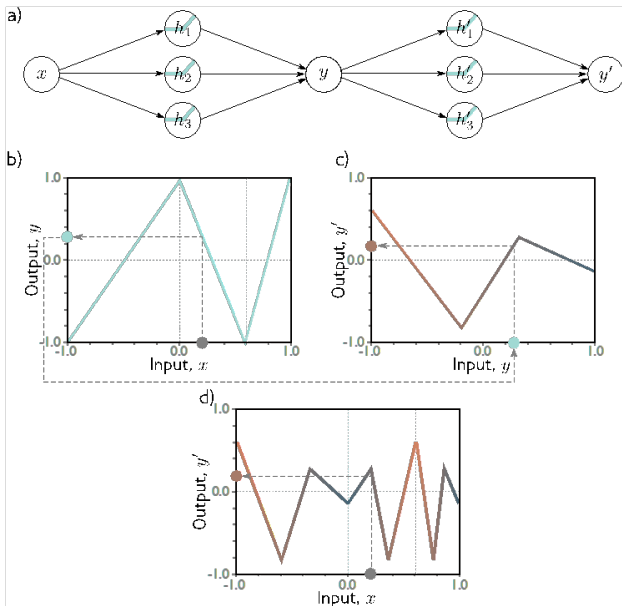
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



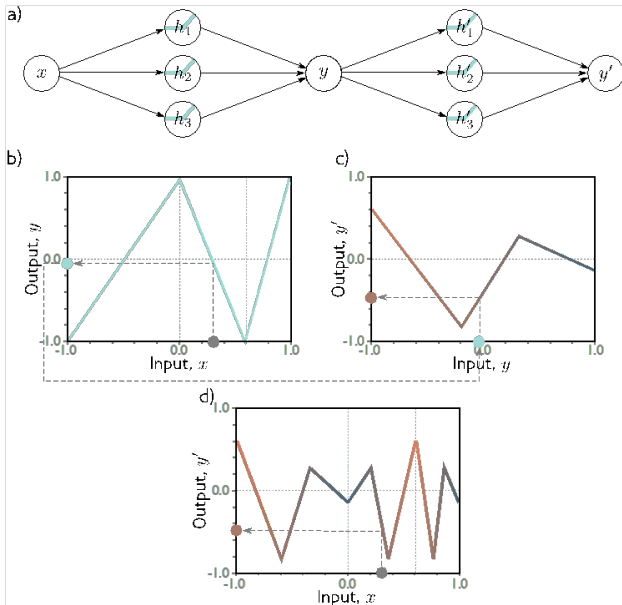
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



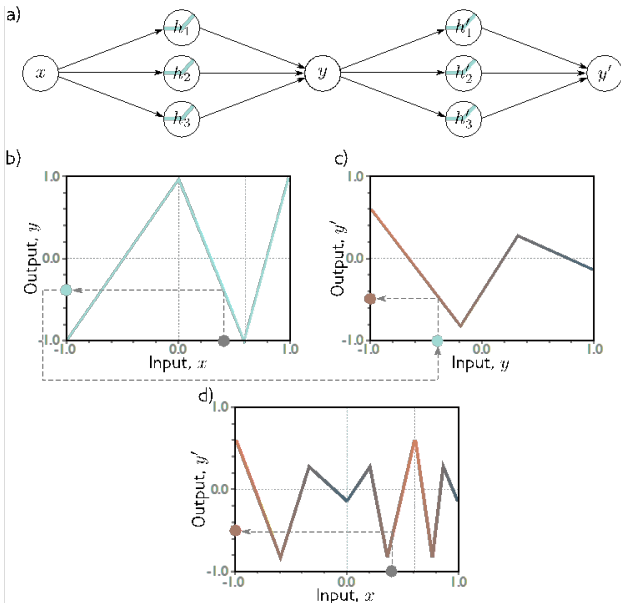
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



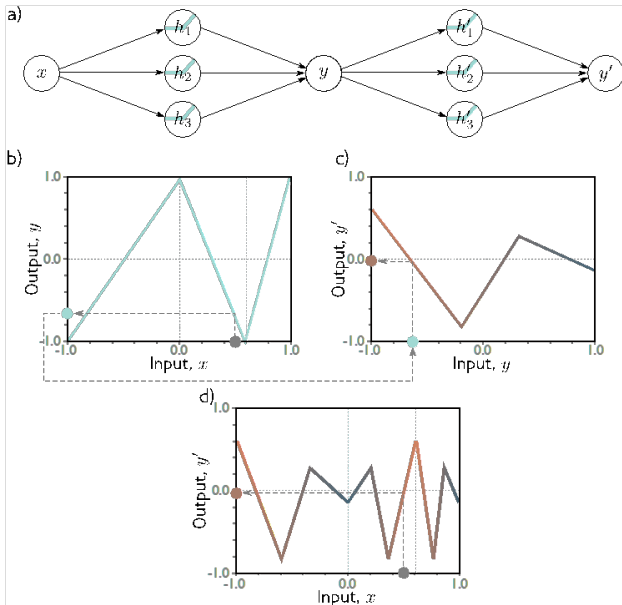
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

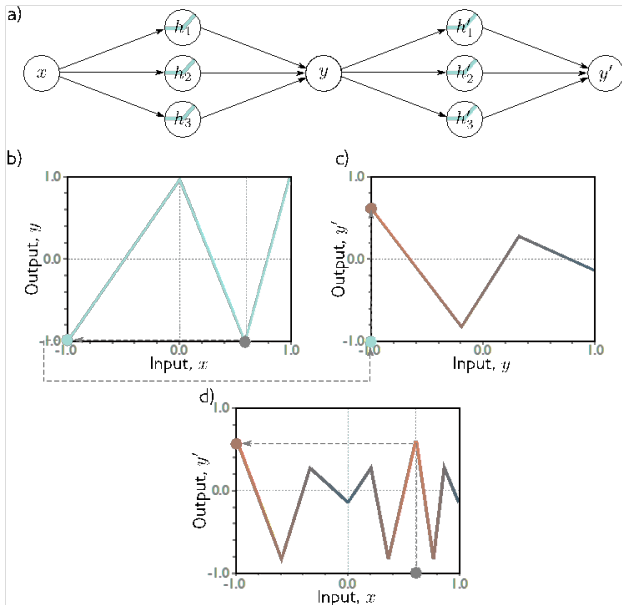
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

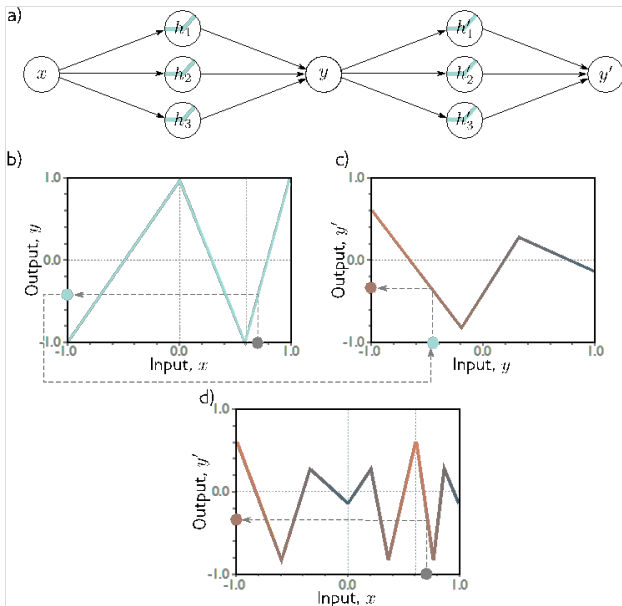
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

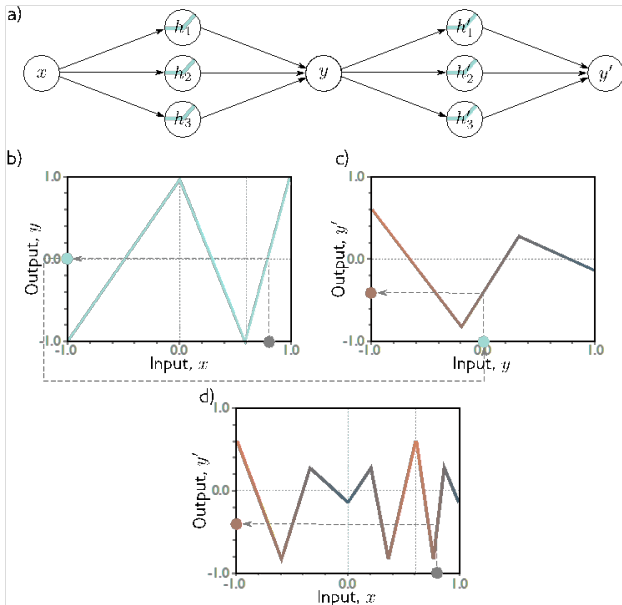
Avaluació de
Models

Generalització

Connexió
amb Xarxes

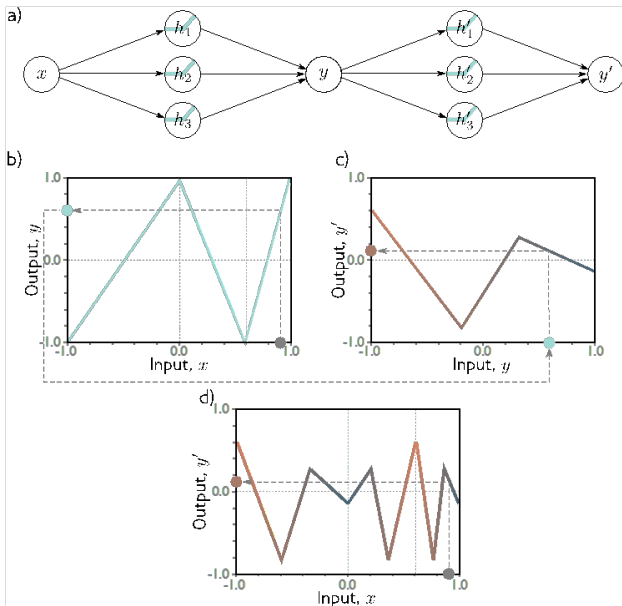
Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Concatenem xarxes shallow

Considerem dues xarxes shallow 1-3-1



Analògia del plegament

Aprentatge
Automàtic II:
Fonaments
de l'Aprentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

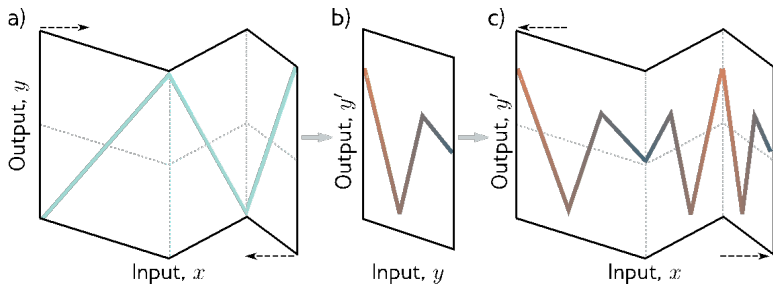
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Considerem dues xarxes shallow 1-3-1. Una xarxa es plega sobre l'altra!



Escrivim 1-3-1-3-1

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

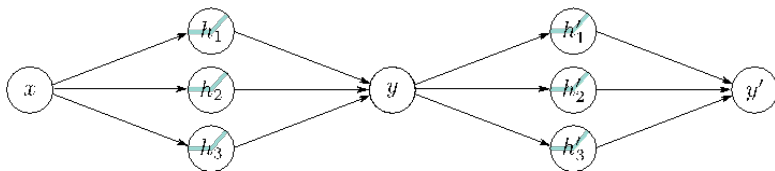
Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Considerem dues xarxes *shallow* idèntiques amb arquitectura 1-3-1 tal que l'output y de la primera xarxa (A) és l'input de la segona xarxa (B).



Primera xarxa, A, y : Entrada $x \in \mathbb{R}^1$, sortida $y = h_A^{(2)} \in \mathbb{R}^1$.

- $\Theta_A^{(1)} \in \mathbb{R}^{3 \times 1}$, $\theta_{0,A}^{(1)} \in \mathbb{R}^3$

- $\Theta_A^{(2)} \in \mathbb{R}^{1 \times 3}$, $\theta_{0,A}^{(2)} \in \mathbb{R}^1$

$$y = h_A^{(2)} = a^{(2)}(\Theta_A^{(2)} a^{(1)}(\Theta_A^{(1)} x + \theta_{0,A}^{(1)}) + \theta_{0,A}^{(2)})$$

Segona xarxa, B, y' : L'entrada de B és la sortida d'A: $x_B = y = h_A^{(2)}$.

- $\Theta_B^{(1)} \in \mathbb{R}^{3 \times 1}$, $\theta_{0,B}^{(1)} \in \mathbb{R}^3$

- $\Theta_B^{(2)} \in \mathbb{R}^{1 \times 3}$, $\theta_{0,B}^{(2)} \in \mathbb{R}^1$

$$y' = a^{(2)}(\Theta_B^{(2)} a^{(1)}(\Theta_B^{(1)} h_A^{(2)} + \theta_{0,B}^{(1)}) + \theta_{0,B}^{(2)})$$

Connexió entre xarxes shallow

Quan fem la concatenació tal que la sortida de la primera xarxa (A) y es correspon amb l'input de la segona xarxa (B), vol dir que $a_A^{(2)}(z^{(2)}) = z^{(2)}$

$$\mathbf{z}_B^{(1)} = \Theta_B^{(1)} \underbrace{a_A^{(2)}(\Theta_A^{(2)} \mathbf{h}_A^{(1)} + \theta_{0,A}^{(2)})}_{y = \mathbf{h}_A^{(2)}} + \theta_{0,B}^{(1)} = \Theta_B^{(1)} (\Theta_A^{(2)} \mathbf{h}_A^{(1)} + \theta_{0,A}^{(2)}) + \theta_{0,B}^{(1)}$$

Aleshores:

$$\mathbf{z}_B^{(1)} = \underbrace{(\Theta_B^{(1)} \Theta_A^{(2)})}_{\Theta_{deep}^{(2)}} \mathbf{h}_A^{(1)} + \underbrace{(\Theta_B^{(1)} \theta_{0,A}^{(2)} + \theta_{0,B}^{(1)})}_{\theta_{0,deep}^{(2)}}$$

On $\Theta_{deep}^{(2)} \in \mathbb{R}^{3 \times 3}$, ja que $(3 \times 1) \times (1 \times 3) = (3 \times 3)$.

L'expressió final de la xarxa concatenada es pot reescriure:

❶ $\mathbf{h}^{(0)} = x$

❷ $\mathbf{h}^{(1)} = a^{(1)}(\Theta^{(1)} \mathbf{h}^{(0)} + \theta_0^{(1)})$ (Capa oculta 1)

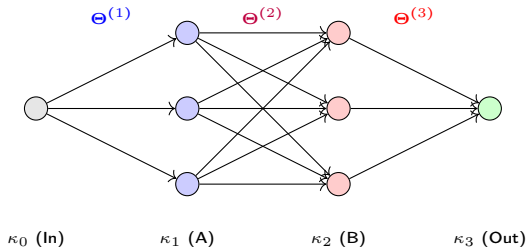
❸ $\mathbf{h}^{(2)} = a^{(2)}(\Theta^{(2)} \mathbf{h}^{(1)} + \theta_0^{(2)})$ (Capa oculta 2 - Fusió)

❹ $y' = a^{(3)}(\Theta^{(3)} \mathbf{h}^{(2)} + \theta_0^{(3)})$ (Capa de sortida)

Equivalència: Dues Shallow = Una Deep

La concatenació resultant és una xarxa amb 3 capes de càlcul ($K = 3$) i arquitectura 1-3-3-1:

- **Capa 1:** $\Theta^{(1)} = \Theta_A^{(1)} \in \mathbb{R}^{3 \times 1}$ i $\theta_0^{(1)} = \theta_{0,A}^{(1)} \in \mathbb{R}^3$.
- **Capa 2:** $\Theta^{(2)} = \Theta_B^{(1)} \Theta_A^{(2)} \in \mathbb{R}^{3 \times 3}$ i $\theta_0^{(2)} = \Theta_B^{(1)} \theta_{0,A}^{(2)} + \theta_{0,B}^{(1)} \in \mathbb{R}^3$.
- **Capa 3:** $\Theta^{(3)} = \Theta_B^{(2)} \in \mathbb{R}^{1 \times 3}$ i $\theta_0^{(3)} = \theta_{0,B}^{(2)} \in \mathbb{R}^1$.



Arquitectura Deep: 1-3-3-1

Quant “costa” a nivell de paràmetres $\mathcal{P}$ una xarxa 1-3-3-1?

- **Capa 1 (κ_1):** $\Theta^{(1)} \in \mathbb{R}^{3 \times 1}$ (3 pesos) + $\theta_0^{(1)} \in \mathbb{R}^3$ (3 biaixos) = **6 paràmetres.**
- **Capa 2 (κ_2):** $\Theta^{(2)} \in \mathbb{R}^{3 \times 3}$ (9 pesos) + $\theta_0^{(2)} \in \mathbb{R}^3$ (3 biaixos) = **12 paràmetres.**
- **Capa 3 (κ_3):** $\Theta^{(3)} \in \mathbb{R}^{1 \times 3}$ (3 pesos) + $\theta_0^{(3)} \in \mathbb{R}^1$ (1 biaix) = **4 paràmetres.**

Total Paràmetres $\mathcal{P}_{deep}$: $6 + 12 + 4 = 22$

Capacitat de Representació (Regions)

Sabem que una xarxa amb una capa oculta de D neurones pot crear fins a $D + 1$ regions lineals. En apilar capes, les regions es multipliquen:

- A l'exemple hem vist una regió de x tal que teníem 3 regions lineals a la primera capa i 3 regions lineals a la segona: $\mathcal{R} = 3 \times 3 = 9$ regions.
- La primera capa pot crear fins a $3 + 1 = 4$ regions $\implies$
Potencialment podem crear $\mathcal{R} = (D^{(1)} + 1) \times (D^{(2)} + 1) = 16$ regions.

Cas general per un únic input i $K - 1$ capes ocultes:

$$\mathcal{R}_{max} \leq \prod_{l=1}^{K-1} (D^{(l)} + 1)$$

Comparativa: Shallow (1-6-1) vs Deep (1-3-3-1)

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

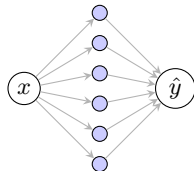
Generalització

Connexió amb Xarxes

Considerem ara una xarxa *shallow* amb el mateix nombre total de neurones ocultes (6):

Xarxa Shallow 1-6-1:

- $\Theta^{(1)} \in \mathbb{R}^{6 \times 1} \rightarrow 6$ pesos.
- $\theta_0^{(1)} \in \mathbb{R}^6 \rightarrow 6$ biaixos.
- $\Theta^{(2)} \in \mathbb{R}^{1 \times 6} \rightarrow 6$ pesos.
- $\theta_0^{(2)} \in \mathbb{R}^1 \rightarrow 1$ biaix.



Total $\mathcal{P}_{shallow}$: $6 + 6 + 6 + 1 = 19$

Model	Neurones Ocultes	Paràmetres	Regions Màx.
Shallow 1-6-1	6	19	$6 + 1 = 7$
Deep 1-3-3-1	6	22	16

- **Eficiència:** Amb només 3 paràmetres més, hem **més que duplicat** la resolució del model (de 7 a 16).
- **Generalització:** Per obtenir 16 regions amb una xarxa *shallow*, necessitaríem 15 neurones ocultes (≈ 46 paràmetres), el doble que la *deep*.

Conclusió: La profunditat permet "reutilitzar" els talls de les capes anteriors, podent representar una major complexitat.

L'Avantatge Deep: Composició de Funcions

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

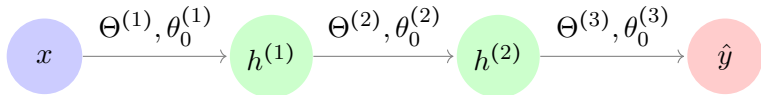
Aprenentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes



Què està passant realment?

La xarxa no aprèn una funció complexa de cop (Shallow), sinó que **compon funcions senzilles**:

$$\hat{y} = f^{(3)}(f^{(2)}(f^{(1)}(x)))$$

Cada capa l realitza una transformació afí seguida d'una no-linealitat $a(\cdot)$:

$$h^{(l)} = a(\Theta^{(l)}h^{(l-1)} + \theta_0^{(l)})$$

Deep Learning: Generalització Multicapa

Aprenentatge Automàtic II: Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprenentatge Supervisat

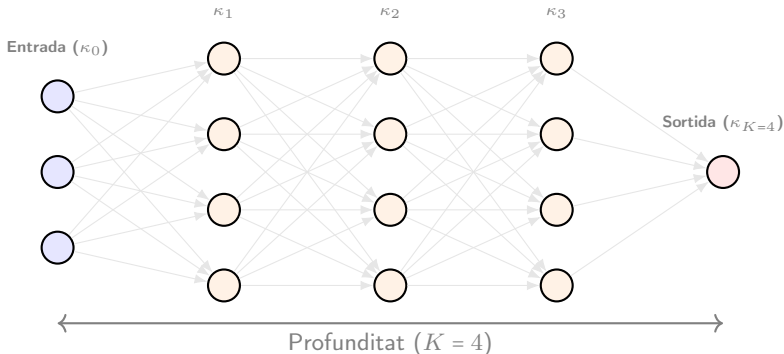
Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

Una xarxa **profunda** (Deep Neural Network) és una composició jeràrquica de funcions. Cada capa κ_l extreu representacions més abstractes que l'anterior.



- **Capa a capa:** Cada capa κ_l rep les característiques de la capa anterior κ_{l-1} i les transforma en una representació més abstracta.

- **Composició de funcions:**

$$\hat{y} = a^{(K)}(\Theta^{(K)}(\dots a^{(1)}(\Theta^{(1)}\mathbf{x} + \theta_0^{(1)})\dots) + \theta_0^K).$$

Per a una xarxa amb K capes (la capa input κ_0 no compta), la computació neurona a neurona és ineficient. Adoptem la **notació vectorial** per a cada capa l :

- $\mathbf{h}^{(l-1)} \in \mathbb{R}^{D^{(l-1)}}$: Vector d'activacions de la capa anterior (entrada).
- $\mathbf{z}^{(l)} \in \mathbb{R}^{D^{(l)}}$: Vector de sumes ponderades (pre-activació).
- $\Theta^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l-1)}}$: Matriu de pesos per a la capa l .
- $\theta_0^{(l)} \in \mathbb{R}^{D^{(l)}}$: Vector de biaixos per a la capa l .

Pas cap endavant (Forward pass) de forma compacta:

$$\mathbf{z}^{(l)} = \Theta^{(l)} \mathbf{h}^{(l-1)} + \theta_0^{(l)} \implies \mathbf{h}^{(l)} = a^{(l)}(\mathbf{z}^{(l)})$$

Per què és més còmode?

- **Eficiència:** Permet l'ús d'àlgebra lineal optimitzada (GPUs).
- **Backpropagation:** El càlcul del gradient $\frac{\partial L}{\partial \Theta^{(l)}}$ esdevé un producte de matrius (regla de la cadena vectorial), facilitant la implementació algorísmica.

En una xarxa neuronal profunda (Deep Learning), els paràmetres s'organitzen per capes:

- **Matrius de Pesos (Θ):** Cada salt entre la capa $l - 1$ i la capa l genera una matriu:

$$\Theta^{(l)} \in \mathbb{R}^{\text{files} \times \text{columnes}} = \mathbb{R}^{D^{(l)} \times D^{(l-1)}}$$

- **Vectors de Biaix (θ_0):** Cada neurona de cada capa oculta i de sortida té el seu propi biaix independent:

$$\theta_0^{(l)} \in \mathbb{R}^{D^{(l)}}$$

Estructura d'una Xarxa Neuronal Completa

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Una xarxa amb K capes de càlcul es compon de:

- **Capa d'Entrada (κ_0):** No és una capa de càlcul. Simplement representa les dades d'entrada $\mathbf{x} \in \mathbb{R}^{D^{(0)}}$, on $D^{(0)}$ és el nombre de característiques d'entrada. Definim la seva "activació" com la pròpia entrada:

$$\mathbf{h}^{(0)} = \mathbf{x}$$

- **Capes Ocultes ($\kappa_1, \dots, \kappa_{K-1}$):** Són les capes intermèdies on es realitza la major part del processament i l'aprenentatge.
 - Per a una capa oculta l , la seva entrada és la sortida de la capa anterior, $\mathbf{h}^{(l-1)}$, que té una dimensió $D^{(l-1)}$.
 - Si la capa l té $D^{(l)}$ neurones, la seva matriu de pesos $\Theta^{(l)}$ tindrà dimensions $D^{(l)} \times D^{(l-1)}$ i el seu vector de biaixos $\theta_0^{(l)}$ tindrà dimensió $D^{(l)}$.
 - El càlcul de les seves activacions es regeix per:

$$\mathbf{h}^{(l)} = a^{(l)}(\Theta^{(l)}\mathbf{h}^{(l-1)} + \theta_0^{(l)})$$

- **Capa de Sortida (κ_K):** És l'última capa i produeix la predicció final del model, $\hat{\mathbf{y}}$. El seu càlcul és idèntic al d'una capa oculta, rebent com a entrada les activacions de l'última capa oculta, $\mathbf{h}^{(K-1)}$:

$$\hat{\mathbf{y}} = \mathbf{h}^{(K)} = a^{(K)}(\Theta^{(K)}\mathbf{h}^{(K-1)} + \theta_0^{(K)})$$

Quan dissenyem una xarxa, hem de definir la seva **topologia** mitjançant paràmetres que no s'aprenen durant l'entrenament: els **hiperparàmetres**.

- Profunditat (*Depth*): El nombre total de capes amb pesos (L). Una xarxa és "Deep" si $K \geq 2$.
- Amplada (*Width*): El nombre de neurones en una capa determinada ($D^{(l)}$). Sol ser constant o decreixer cap a la sortida.
- Capacitat: Relacionat amb el nombre total de paràmetres entrenables (pesos i biaixos).

Càlcul General de Paràmetres Lliures

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

El nombre total de paràmetres lliures ($\mathcal{P}$) d'una xarxa neuronal densa és la suma dels pesos i biaixos de cada una de les seves capes.

Fórmula General

Si definim $D^{(l)}$ com el nombre de neurones de la capa l , per a una xarxa amb L capes (sense comptar l'entrada κ_0):

$$\mathcal{P} = \sum_{l=1}^L \text{paràmetres de la capa } \kappa_l = \sum_{l=1}^L \underbrace{D^{(l)}(D^{(l-1)} + 1)}_{\text{pesos} + \text{biaixos}}$$

Components de la fórmula:

- $D^{(l)} \times D^{(l-1)}$: Nombre de connexions sinàptiques (pesos) que arriben a la capa l .
- $D^{(l)}$: Nombre de biaixos (un per cada neurona de la capa l).

Nota sobre el Deep Learning: A mesura que afegim capes ($K \uparrow$), la riquesa del model creix, però també el risc de *overfitting*, per la qual cosa el control de $\mathcal{P}$ és crític per al disseny de l'arquitectura.

Exercici: Paràmetres per a una xarxa 1-2-2-1

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

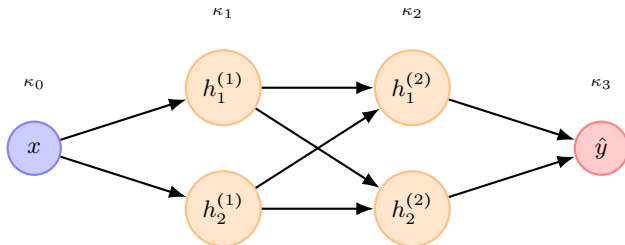
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal (dues capes ocultes)?



Paràmetres per a una xarxa 1-3-3-2

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

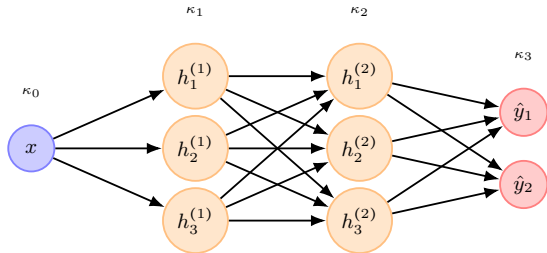
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal (dues capes ocultes)?
Escriu les matrius de pesos i biaixos a cada capa.



Paràmetres per a una xarxa 3-4-4-2

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

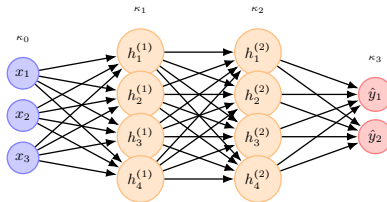
Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Quants paràmetres lliures té aquesta xarxa neuronal? Escribeu les matrius de pesos i biaixos a cada capa.



Exercici: Anatomia d'una xarxa Deep

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Considerem una xarxa neuronal per a classificació d'imatges amb arquitectura **784-100-50-10**. Calcula el nombre total de paràmetres entrenables:

Exercici

Considera una xarxa neuronal amb un sol input i un sol output amb K capes ($K - 1$ capes ocultes) amb D neurones a cada capa. Demostra que el nombre total de paràmetres és

$$\mathcal{P} = 3D + 1 + (K - 2)D(D + 1),$$

i compara-ho amb el nombre de paràmetres de una xarxa shallow 1-D-1.

Exemple de Forward Pass i Loss (1-2-2-1)

Considerem una entrada x . El càlcul de la sortida $\hat{y}$ es realitza capa a capa:

❶ **Capa 1:** $\mathbf{z}^{(1)} = \Theta^{(1)}x + \theta_0^{(1)} \implies \mathbf{h}^{(1)} = a^{(1)}(\mathbf{z}^{(1)}) \in \mathbb{R}^2$

❷ **Capa 2:** $\mathbf{z}^{(2)} = \Theta^{(2)}\mathbf{h}^{(1)} + \theta_0^{(2)} \implies \mathbf{h}^{(2)} = a^{(2)}(\mathbf{z}^{(2)}) \in \mathbb{R}^2$

❸ **Capa 3:** $z^{(3)} = \Theta^{(3)}\mathbf{h}^{(2)} + \theta_0^{(3)} \implies \hat{y} = a^{(3)}(z^{(3)}) \in \mathbb{R}^1$

Càlcul de l'Error (Loss Function): Un cop tenim la predicció $\hat{y}$, la comparem amb el valor real y (el *ground truth*). Si fem servir MSE per a una sola mostra:

$$L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

L'objectiu de l'aprenentatge

Volem trobar el conjunt de paràmetres $\mathcal{W} = \{\Theta^{(l)}, \theta_0^{(l)}\}$ que **minimitzi** aquesta L . Per fer-ho, calcularem el gradient de la loss L respecte a cada paràmetre usant la regla de la cadena (**Backpropagation**).

Exercici numèric de Forward Pass (1-2-2-1)

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Considereu una entrada $x = 1$ i el valor real $y = 5$ i funcions d'activació lineals. Calculeu la predicció $\hat{y}$ i el MSE:

❶ **Capa 1** ($1 \rightarrow 2$): $\Theta^{(1)} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \theta_0^{(1)} = \begin{pmatrix} 0.5 \\ 1 \end{pmatrix}$

❷ **Capa 2** ($2 \rightarrow 2$): $\Theta^{(2)} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \theta_0^{(2)} = \begin{pmatrix} 0 \\ -0.5 \end{pmatrix}$

❸ **Capa 3** ($2 \rightarrow 1$): $\Theta^{(3)} = \begin{pmatrix} 1 & 1 \end{pmatrix}, \theta_0^{(3)} = 0$

Exercici numèric de Forward Pass (1-3-3-2)

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Considereu una entrada $x = 2$ i el valor real $y = [10, 5]^T$ amb funcions d'activació lineals. Calculem la predicció $\hat{y}$ i el MSE:

❶ Capa 1 ($1 \rightarrow 3$): $\Theta^{(1)} = \begin{pmatrix} 1 \\ 0.5 \\ 0 \end{pmatrix}, \theta_0^{(1)} = \begin{pmatrix} 0 \\ 1 \\ 2 \end{pmatrix}$

❷ Capa 2 ($3 \rightarrow 3$): $\Theta^{(2)} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}, \theta_0^{(2)} = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}$

❸ Capa 3 ($3 \rightarrow 2$): $\Theta^{(3)} = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \theta_0^{(3)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

Nombre de regions

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

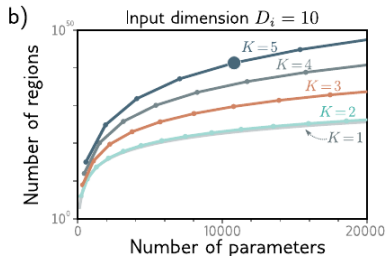
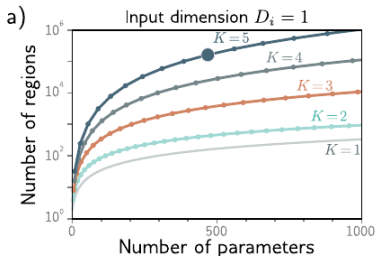
Avaluació de
Models

Generalització

Connexió
amb Xarxes

Per al cas particular d'una xarxa profunda amb $D^{(0)}$ inputs i $D^{(1)} = D^{(2)} = \dots = D^{(K-1)} \equiv D$ neurones a les capes ocultes, el nombre màxim de regions lineals està acotat per la cota superior:

$$\mathcal{R} \leq \left(\frac{D}{D^{(0)}} + 1 \right)^{D^{(0)}(K-2)} \sum_{j=0}^{D^{(0)}} \binom{D}{j}$$



Comparativa d'Eficiència: Shallow vs. Deep

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Si disposem d'un pressupost de **6 neurones ocultes**, comparem la capacitat de fragmentació ($\mathcal{R}$) segons l'arquitectura triada:

1. Xarxa Shallow (1-6-1)

- $N = 6, D^{(0)} = 1$
- Calculem:

$$\mathcal{R} = \sum_{j=0}^1 \binom{6}{j} = 1 + 6 = 7$$

- **Eficiència:** Creixement lineal.

2. Xarxa Deep (1-3-3-1)

- $D = 3, K = 3, D^{(0)} = 1$
- **Càlcul Cota Superior:**

$$\mathcal{R} = \underbrace{(3+1)^{3-2}}_{\text{Plegament}} \times \underbrace{\sum_{j=0}^1 \binom{3}{j}}_{\text{Capa 1}}$$

$$\mathcal{R} = 4^1 \times 4 = 16$$

Arquitectura	Paràmetres ($\mathcal{P}$)	Regions ($\mathcal{R}$)	Ratio $\mathcal{R}/\mathcal{P}$
Shallow (1-6-1)	19	7	0.36
Deep (1-3-3-1)	22	16	0.72
Very Deep (1-2-2-2-1)	21	27	1.28

Conclusió Final

Amb pràcticament els mateixos paràmetres (≈ 20), l'arquitectura més profunda genera gairebé **4 vegades més regions** que la shallow!

L'Eficiència del Deep Learning

Si disposem d'un pressupost fix de $N = 100$ **neurons ocultes**, quina arquitectura és més eficient per fragmentar l'espai d'entrada?

Arquitectura	Configuració	Càlcul de Regions (1D)	Total Regions
Shallow	1-100-1	$100 + 1$	101
Deep (2 capes)	1-50-50-1	51×51	2.601
Deep (10 capes)	1-10-10-...-1	11^{10}	$\approx 25 \times 10^9$

Conclusió Lògica

Mentre que en una xarxa **Shallow** la capacitat creix de forma **additiva** amb el nombre total de neurones, en una xarxa **Deep** la capacitat creix de forma **multiplicativa** (D^{K-2}).

Per què és més eficient?

- El Deep Learning permet crear funcions d'una complexitat immensa amb un nombre molt reduït de paràmetres.
- Per obtenir les 11^{10} regions amb una xarxa shallow, ens caldrien milers de milions de neurones (i de memòria RAM!).

Deep Learning en el món real: Cas MNIST

Aprentatge Automàtic II: Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Context: AI, ML i DL

Taxonomia del Machine Learning

Aprentatge Supervisat

Inferència Estadística: Pèrdua i Versemblança

Avaluació de Models

Generalització

Connexió amb Xarxes

En MNIST, l'entrada té dimensió $D^{(0)} = 784$. Considerem ara un pressupost de $N = 200$ neurones per aquest problema i comparem una xarxa *shallow* vs. *deep*:

- **Cas Shallow (784-200-10):**

$$\mathcal{R}_S = \sum_{j=0}^{784} \binom{200}{j} = 2^{200} \approx 1.6 \times 10^{60}$$

(Nota: Com que $N < D^{(0)}$, la suma és 2^N).

- **Cas Deep (784-100-100-10):** Repartim les 200 neurones en dues capes de $D = 100$. Ara $K = 3$ (sortida en $l = 3$).

$$\mathcal{R}_D = \left(\frac{100}{784} + 1 \right)^{784(3-2)} \times \sum_{j=0}^{100} \binom{100}{j} \approx (2.8 \times 10^{40}) \times (2^{100}) \approx 3.3 \times 10^{70}$$

(Nota: El sumatori ens dona 2^{100} (regions de la primera capa), i el factor multiplicatiu és $(1.127)^{784} \approx 7.5 \times 10^{40}$).

Conclusió MNIST

En alta dimensió, el Deep Learning no només afegeix regions, sinó que les **multiplica per ordres de magnitud** (10^{10} vegades més regions en aquest exemple).

Xarxes deep i entrenament

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Context: AI,
ML i DL

Taxonomia
del Machine
Learning

Aprenentatge
Supervisat

Inferència
Estadística:
Pèrdua i Ver-
semblança

Avaluació de
Models

Generalització

Connexió
amb Xarxes

Figure 20.2 MNIST-1D training. Four fully connected networks were fit to 4000 MNIST-1D examples with random labels using full batch gradient descent, He initialization, no momentum or regularization, and learning rate 0.0025. Models with 1,2,3,4 layers had 298, 100, 75, and 63 hidden units per layer and 15208, 15210, 15235, and 15139 parameters, respectively. All models train successfully, but deeper models require fewer epochs.

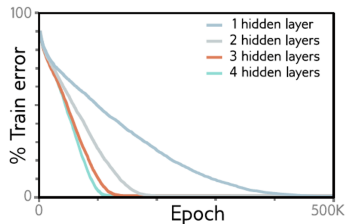
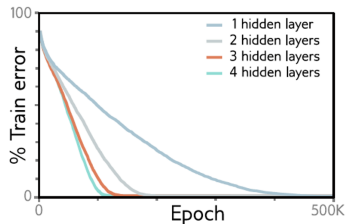


Figure 20.2 MNIST-1D training. Four fully connected networks were fit to 4000 MNIST-1D examples with random labels using full batch gradient descent, He initialization, no momentum or regularization, and learning rate 0.0025. Models with 1,2,3,4 layers had 298, 100, 75, and 63 hidden units per layer and 15208, 15210, 15235, and 15139 parameters, respectively. All models train successfully, but deeper models require fewer epochs.



Entrenament

- Però com s'entrenen les xarxes? Com trobem els pesos i biaixos que ens minimitzen la loss function en cada cas?